作品编号：CACR2026T4PMR7

作品类别：☑软件设计　☐硬件制作　☐工程实践

☐密码应用技术　☐其它

# 2026 年第十一届全国密码技术竞赛作品设计报告

作品题目：基于多类格困难假设的抗量子身份基匹配加密通用设计与优化实现

2026 年 10 月

中国密码学会

# 基本信息表

编号：CACR2026T4PMR7

作品题目：基于多类格困难假设的抗量子身份基匹配加密通用设计与优化实现

作品类别：☑软件设计 ☐硬件制作 ☐工程实践 ☐密码应用技术 ☐其它

**作品内容摘要：**

量子计算的发展对基于整数分解和离散对数问题的传统公钥密码提出了新的安全挑战，面向后量子环境的密码迁移已经成为网络与信息系统建设的重要问题。格密码具有多样的困难问题和丰富的结构形式，是抗量子公钥密码的重要技术路线。与此同时，证书体系下的加密与签名通常分别完成接收者访问控制和发送者身份认证；在机构定向投递、受控设备通信和来源受限的消息服务中，系统还需要在一次消息处理过程中同时核对通信双方的身份。身份基匹配加密正是面向这一需求：只有接收者身份与发送者指定的身份一致，并且发送者身份与接收者认可的身份一致时，消息才能被恢复。

本作品基于两层匿名分层身份基加密（HIBE）与身份基签名（IBS）的通用构造，设计并实现一套基于多类格困难假设的抗量子身份基匹配加密系统。系统将 HIBE 与 IBS 封装为彼此独立、可替换的密码后端，通过统一接口、能力声明和兼容性检查完成组合。HIBE 只封装 32 字节随机种子，应用消息由该种子派生的 AES-256-GCM 密钥保护，从而适配不同后端的明文容量；IBS 负责绑定发送身份和一次性验签公钥，外层一次性签名进一步认证完整密文字节。系统还实现了规范对象编码、固定身份发钥、状态持久化、异常拒绝和重启恢复。

作品完成 LATTE、Lai 两种 HIBE 与 BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli 五种常规 IBS 的十种端到端组合，并将目标规模 Wang–PW IBS 接入 LATTE 组合。针对完整系统运行中暴露的计算与存储瓶颈，作品实现三项重点优化：利用多项式结构和逐频率低秩计算降低 Sageloli 采样的中间矩阵开销；按照字段实际取值范围对 SRGG 公钥、主状态、用户密钥和签名进行无损编码；为固定身份对建立 Lai 接收会话，将重复委派移出逐消息解密路径。实测中，SRGG 三档主状态缩小约 5.05 至 5.46 倍；Lai 会话准备约 510 秒，完成后续解密的中位数为 28.6 秒。

报告选取三组代表组合分析运行时间、内存和通信量。其中，A 组重复加密与解密中位数分别为 78.5 毫秒和 152 毫秒；B 组用于分析原生模格 IBS 的计算与对象开销；C 组用于展示高成本环格方案及会话复用的效果。软件提供命令行与浏览器两种操作入口，能够完成系统建立、双方发钥、消息发送、匹配解密、错误身份拒绝和状态恢复，形成从多类格原语到完整消息应用的统一实现。

**关键词（五个）：**身份基匹配加密；格密码；分层身份基加密；身份基签名；格陷门采样

# 目录

**第一章 作品概述** **1**
1.1 研究背景与意义 . . . 1
1.1.1 量子计算背景下的公钥密码迁移 . . . 1
1.1.2 双方身份约束下的定向通信需求 . . . 1
1.2 国内外研究现状 . . . 2
1.2.1 身份基密码与身份基匹配加密研究 . . . 2
1.2.2 格 HIBE 与格 IBS 研究 . . . 3
1.3 现有方案的工程实现问题 . . . 3
1.4 作品研究内容与技术路线 . . . 4
1.4.1 研究内容 . . . 4
1.4.2 总体技术路线 . . . 4
1.5 主要工作与创新点 . . . 5
1.6 报告结构 . . . 6

**第二章 基础知识与方案依据** **7**
2.1 分层身份基加密 . . . 7
2.2 身份基签名 . . . 8
2.3 格结构与困难问题 . . . 8
2.3.1 LWE 与 SIS 问题 . . . 8
2.3.2 困难问题与系统原语的对应关系 . . . 9
2.4 通用构造的安全目标 . . . 10
2.5 已实现算法与代表组合 . . . 10
2.6 本章小结 . . . 11

**第三章 通用 IB-ME 系统设计** **12**
3.1 参与者、对象与身份顺序 . . . 12
3.2 IB-ME 五算法设计 . . . 13
3.2.1 系统建立算法 Setup . . . 13
3.2.2 发送者密钥生成算法 SenderKeyGen . . . 13
3.2.3 接收者密钥生成算法 ReceiverKeyGen . . . 13
3.2.4 消息加密算法 Encrypt . . . 13
3.2.5 匹配解密算法 Decrypt . . . 14
3.3 基于短种子封装的混合加密 . . . 14
3.4 完整密文字节认证 . . . 15
3.5 协议正确性与异常处理 . . . 15
3.6 代表组合与应用消息容量 . . . 16
3.7 本章小结 . . . 16

**第四章 关键原语实现与工程优化** **17**

4.1 两种分层身份基加密 . . . 17
4.1.1 LATTE 两层身份委派机制 . . . 17
4.1.2 Lai 分层身份块扩展机制 . . . 17
4.2 身份基签名后端 . . . 18
4.2.1 BNN 认证转换与 ML-DSA / Falcon 接入 . . . 18
4.2.2 Luo 模格身份签名 . . . 19
4.2.3 SRGG 身份目标与消息挑战关系 . . . 19
4.2.4 Sageloli 环陷门与非齐次签名方程 . . . 19
4.2.5 Wang–PW 目标规模矩阵 SIS 实现 . . . 20
4.3 Sageloli 结构化环采样优化 . . . 21
4.3.1 稠密矩阵展开的资源瓶颈 . . . 21
4.3.2 逐频率低秩分解方法 . . . 21
4.3.3 数值采样与精确代数校验 . . . 22
4.4 SRGG 对象的范围感知编码 . . . 22
4.4.1 参考表示的空间开销 . . . 22
4.4.2 四类对象的规范编码与解码 . . . 23
4.5 Lai 接收会话优化 . . . 23
4.5.1 重复委派的性能瓶颈 . . . 23
4.5.2 身份绑定与会话生命周期 . . . 24
4.5.3 连续消息的摊销成本 . . . 24
4.6 本章小结 . . . 24

**第五章 软件系统与操作流程 25**
5.1 分层软件架构 . . . 25
5.2 稳定的原语接口 . . . 26
5.3 组合配置与接入流程 . . . 26
5.3.1 完整组合配置 SuiteConfig . . . 26
5.3.2 新增 HIBE 或 IBS 的接入步骤 . . . 27
5.4 固定发钥与持久状态 . . . 27
5.5 命令行与浏览器操作 . . . 28
5.6 运行优化与部署方式 . . . 29
5.7 本章小结 . . . 30

**第六章 测试与结果分析 31**
6.1 实验设置 . . . 31
6.1.1 十种常规组合的覆盖范围 . . . 31
6.1.2 评价指标 . . . 32
6.2 完整系统的时间与存储成本 . . . 32
6.2.1 A/B 组合对比 . . . 32
6.2.2 对象尺寸与运行内存 . . . 33
6.2.3 C 组合的委派开销 . . . 34
6.2.4 Wang–PW 目标档的分阶段运行 . . . 34
6.3 完整密文的通信开销 . . . 35
6.3.1 完整密文的组成 . . . 35

6.3.2 消息长度与可用容量 . . . . . 36
6.4 三项主要优化的实验结果 . . . . . 37
6.4.1 Sageloli 结构化采样结果 . . . . . 37
6.4.2 SRGG 紧凑编码结果 . . . . . 37
6.4.3 Lai 接收会话结果 . . . . . 39
6.5 缓存与部署位置的完整流程对照 . . . . . 39
6.5.1 局部缓存对总耗时的影响 . . . . . 39
6.5.2 文件部署位置对运行时间的影响 . . . . . 39
6.6 功能测试结果 . . . . . 40
6.7 本章小结 . . . . . 41

**第七章 总结与展望** **42**
7.1 工作总结 . . . . . 42
7.2 创新点及实际效果 . . . . . 42
7.3 实验结论 . . . . . 44
7.4 后续工作 . . . . . 44

**参考文献** **45**

# 第一章　作品概述

## 1.1　研究背景与意义

### 1.1.1　量子计算背景下的公钥密码迁移

公钥加密和数字签名是网络通信、身份认证与数据交换的基础。当前广泛使用的 RSA、Diffie–Hellman 和椭圆曲线密码，分别建立在整数分解或离散对数问题之上。Shor 算法表明，具备足够规模的通用量子计算机可以在多项式时间内求解这两类问题 [1]。因此，需要为具有长期保密需求的系统研究不依赖上述困难问题的公钥密码方案，并提前解决算法实现、系统接入和运行评测等迁移问题。

格密码以高维格上的计算问题为基础，目前没有已知的经典或量子多项式时间算法能够求解其适当参数实例。格上误差学习（LWE）、短整数解（SIS）及其环、模和 NTRU 结构变体，能够支持加密、签名、身份基密码等多种功能。不同格结构在密钥尺寸、计算方式、存储形式和工程复杂度上各有特点。研究多类格原语在同一应用协议中的组合方式，有助于比较这些技术路线，也为后续更换算法和参数保留统一的软件基础。

后量子迁移不仅是底层算法的替换，还涉及身份管理和业务协议的重新设计。传统公钥基础设施需要证书把实体身份与公钥绑定，通信端还要完成证书获取、验证、更新和撤销等操作。身份基密码允许直接使用邮箱、机构账号、设备编号等规范身份作为公开标识，由可信发钥机构完成现实身份核验并签发私钥，从而减少通信前的公钥查询与证书传递。对于成员关系明确、由统一机构管理密钥的应用，身份基机制能够使访问控制规则与业务身份直接对应。

### 1.1.2　双方身份约束下的定向通信需求

普通公钥加密主要限制“谁能够读取消息”，数字签名主要证明“消息由谁发送”。在机构定向投递、受控设备指令和来源受限的消息服务中，这两个条件需要同时成立：发送者只允许指定接收者读取，接收者也只接受自己事先认可的发送者。若分别使用加密和签名实现，应用层还需自行规定证书、签名、加密载荷和异常处理的组合顺序，并确保发送身份与接收条件没有在不同处理步骤中失去绑定。

身份基匹配加密（Identity-Based Matchmaking Encryption，IB-ME）把双方身份约束纳入同一次消息恢复过程。记发送者身份为 $s$，发送者指定的接收身份为 $r$；接收密钥所属身份为 $r'$，接收者认可的发送身份为 $s'$。解密成功的必要条件为

$$(r = r') \ \wedge \ (s = s').$$

只有接收身份和认可的发送身份同时匹配，系统才恢复应用消息。表 1.1列出四种身份组合；任意一方不匹配时，接收端均返回拒绝。

**表 1.1** 接收身份与认可发送身份的匹配关系

| 接收身份 | 认可的发送身份 | 处理结果 |
|---|---|---|
| 匹配 | 匹配 | 恢复消息 |
| 匹配 | 不匹配 | 拒绝 |
| 不匹配 | 匹配 | 拒绝 |
| 不匹配 | 不匹配 | 拒绝 |

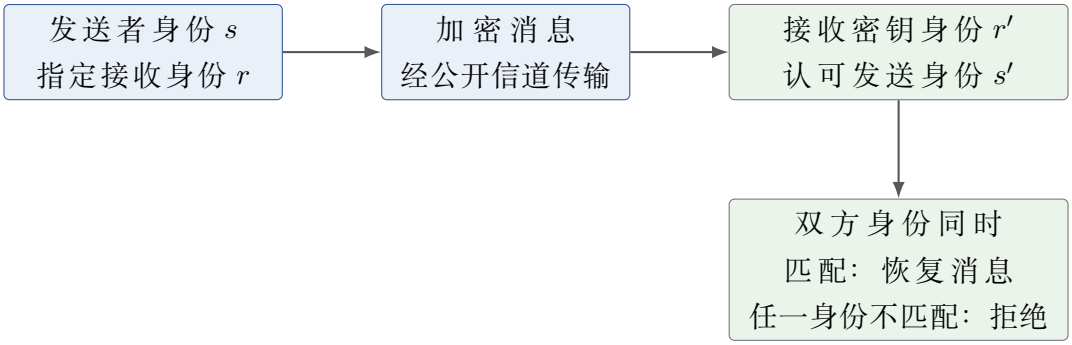



**图 1.1** 身份基匹配加密的双方身份约束

与仅验证接收者身份的加密机制相比，IB-ME 把接收权限和发送来源同时写入密码关系。该机制既可用于机构内部的定向文件投递，也可用于控制终端只执行来自指定管理者的指令。若底层加密同时具备身份匿名性，公开信道上的观察者还不能直接从密文中读取通信双方的身份。因而，在后量子环境下研究 IB-ME 的通用构造和完整软件实现，兼具密码功能与工程应用价值。

## 1.2 国内外研究现状

### 1.2.1 身份基密码与身份基匹配加密研究

Shamir 于 1984 年提出身份基密码的基本思想，使公开身份可以直接参与密码运算 [2]。此后形成了身份基加密（IBE）、分层身份基加密（HIBE）和身份基签名（IBS）等研究方向。HIBE 把单一身份扩展为分层身份路径，并允许上层私钥向下委派；匿名 HIBE 还要求密文不泄露目标身份。IBS 则由可信机构根据用户身份生成签名私钥，验证者使用系统公钥和身份即可检查签名，无需另外传递用户证书。

IB-ME 进一步研究如何在加密阶段和解密阶段分别指定对方身份。Wang 等人给出了由两层匿名 HIBE 与 IBS 构造 IB-ME 的方法，并在格密码上进行了实例化 [3]。在该构造中，HIBE 的两层身份依次对应接收者和发送者，IBS 用于认证发送者及其生成的协议对象。HIBE 和 IBS 可以分别选择具体方案，这为多种格结构的组合提供了理论基础。

现有通用构造主要给出算法关系与安全模型。将其实现为可操作的软件，还需要确定原语接口、对象编码、载荷封装、状态管理和错误处理，并验证不同 HIBE 与 IBS 组合是否满

足相同的身份顺序和消息容量要求。本作品的研究重点正是把该通用构造落实为能够更换后端、保存状态并运行完整消息流程的系统。

### 1.2.2 格 HIBE 与格 IBS 研究

格 HIBE 需要同时处理身份映射、陷门委派和密文匿名性。LATTE 方案利用 NTRU 结构组织两层身份加密，兼顾匿名性与较紧凑的对象表示 [4]；Lai 等人的方案利用理想格陷门完成分层密钥委派 [5]。两种方案在环结构、陷门表示和委派成本上存在明显差异，为比较不同 HIBE 路线提供了基础。

格 IBS 的实现路线更加多样。认证转换可以把 ML-DSA、Falcon 等格签名组件接入身份发钥流程 [9, 10, 18]；Luo 方案直接在模格关系上构造身份签名 [17]；SRGG 和 Sageloli 方案分别利用模格与环格结构缩短对象或提高证明紧致性 [6, 7]；Wang–PW 路线则使用矩阵 SIS 关系和陷门采样实现目标规模身份签名 [3, 8]。这些方案依赖的格结构、参数、密钥形态和签名尺寸并不相同，不能通过复制同一数据结构直接互换。

**表 1.2** 本作品采用的 HIBE 与 IBS 技术路线

| 原语类别 | 采用方案 | 主要格结构 | 在系统中的作用 |
|---|---|---|---|
| HIBE | LATTE、Lai | NTRU 结构格、理想格 | 绑定两层身份并封装随机种子 |
| 常规 IBS | BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli | 模格、NTRU 结构格、环格 | 签署发送身份及协议承诺 |
| 扩展 IBS | Wang–PW | 矩阵 SIS 与格陷门 | 检验大型参数后端的接入与运行 |

上述研究成果提供了完整系统所需的密码组件，但不同论文的接口和实验条件各自独立。要在同一协议下比较多类格路线，需要建立统一的组合边界，并在保持各原语内部参数和对象独立的同时，规范它们与公共协议之间交换的信息。这也是本作品“通用设计”的主要出发点。

## 1.3 现有方案的工程实现问题

从通用构造到完整软件系统，主要存在以下四类工程问题。

**原语接口与对象表示存在差异**。两种 HIBE 和六类 IBS 分别使用矩阵、多项式、模向量和 NTRU 对象，参数版本、身份长度、随机采样和序列化方法也不同。若公共协议直接读取后端的矩阵或陷门字段，每增加一种算法都需要修改协议代码，组合数量增加后也难以保证处理逻辑一致。

**HIBE 明文容量与 IBS 签名尺寸不一致**。部分 HIBE 只适合加密几十字节，而 IBS 签名可能达到数 KiB 甚至更大，应用消息本身也没有固定长度。将“签名 + 消息”直接作为 HIBE

明文，会使不少组合超过容量上限。因此，完整系统必须把身份绑定与大载荷加密分开处理，同时保证两部分不能被替换或重新组合。

**论文参数下的计算和存储开销较高**。论文中的矩阵平方根、Gaussian 采样和陷门委派步骤，若按稠密矩阵直接实现，可能产生数十 GiB 的中间对象。通用 JSON 虽然便于调试，却会重复保存大量整数的符号、分隔符和十进制字符。实现既要保持原有代数关系，又要避免不必要的矩阵展开和对象膨胀。

**单次算法实验与连续消息服务的成本模型不同**。系统建立、用户发钥和固定身份对的叶密钥委派都可以被后续消息复用。若每条消息均重新执行这些步骤，测试结果会被准备成本主导，也无法反映实际服务中的连续收发过程。软件需要区分长期状态、会话状态和逐消息对象，并分别统计首次、重复和恢复后的运行成本。

这四类问题分别对应本作品的统一接口与兼容性检查、短种子混合加密、结构化采样与紧凑编码、接收会话与持久状态设计。后续章节将按照“公共协议—关键原语—软件系统—实验结果”的顺序展开。

## 1.4 作品研究内容与技术路线

### 1.4.1 研究内容

本作品围绕通用协议、密码原语、工程优化和软件验证四个层次开展工作。

第一，依据两层匿名 HIBE 与 IBS 的通用构造，定义系统建立、发送者发钥、接收者发钥、消息加密和匹配解密五个公共算法。协议固定身份顺序和认证上下文，不读取 HIBE 或 IBS 内部的矩阵、陷门与采样状态。

第二，分别实现 LATTE、Lai 两种 HIBE，以及 BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli 五种常规 IBS，形成十种端到端组合。在此基础上，进一步接入目标规模 Wang–PW IBS，并完成其与 LATTE 的完整消息流程。

第三，针对实测瓶颈实现 Sageloli 结构化采样、SRGG 范围感知编码和 Lai 接收会话。三项工作分别降低大型中间矩阵开销、序列化空间开销和固定身份对的重复委派时间。

第四，建设规范对象、状态持久化、命令行和浏览器操作入口，并对系统建立、双方发钥、首次与重复消息收发、错误身份、密文篡改、对象尺寸和峰值内存进行测试。

### 1.4.2 总体技术路线

系统输入包括 HIBE 选择、IBS 选择、接收身份、发送身份和应用消息。装配层首先读取两个后端的能力声明，检查身份长度、明文容量、签名上限和编码版本，再建立一套完整组合。协议层始终调用统一接口，底层参数和秘密状态仅由相应后端管理。

发送端生成 32 字节随机种子，并使用两层 HIBE 沿 $(r, s)$ 身份路径封装该种子；随后通过 HKDF 派生对称密钥，使用 AES-256-GCM 保护 IBS 签名和应用消息。IBS 签名同时绑定发送身份与一次性验签公钥承诺，外层一次性签名认证 HIBE 胶囊、对称密文和公开上下文。接收端依次完成外层认证、HIBE 解封装、GCM 认证、载荷解析、IBS 验签和承诺核对，任一步失败均不输出应用消息。

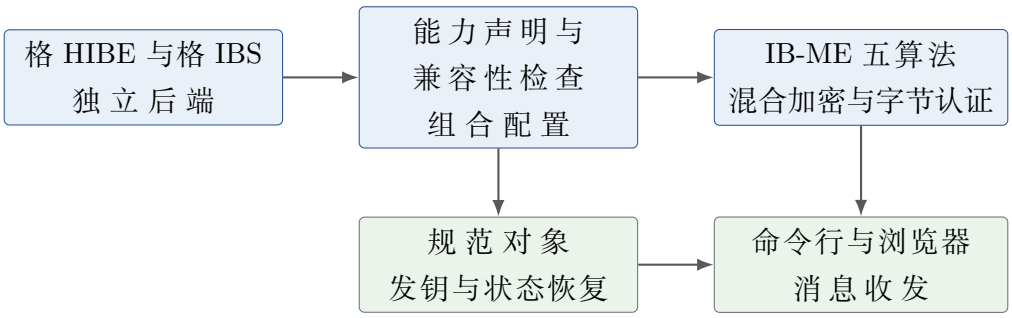



**图 1.2** 作品总体技术路线

该技术路线把“多类格困难假设”落实为相互独立的密码后端，把“通用设计”落实为稳定接口和组合检查，把“优化实现”落实为面向采样、编码和委派的具体改进。各层之间通过规范对象连接，因而新增 HIBE 或 IBS 时无需改写完整消息协议。

## 1.5 主要工作与创新点

作品的主要工作和创新性体现在通用组合、关键原语优化与完整软件实现三个方面。

**表 1.3** 作品的主要工作及其作用

| 工作 | 具体内容 | 解决的问题 |
|---|---|---|
| 通用 IB-ME 设计 | 独立 HIBE / IBS 接口、五算法、32 字节种子封装、完整字节认证、规范对象 | 使不同格结构和明文容量的原语进入同一协议 |
| 多算法组合实现 | 两种 HIBE、五种常规 IBS 的十种配对；Wang–PW 目标档与 LATTE 组合 | 统一实现并比较多类格密码技术路线 |
| 关键算法优化 | Sageloli 结构化采样、SRGG 范围感知编码、Lai 接收会话 | 降低矩阵展开、对象存储和重复委派成本 |
| 软件与实验系统 | 固定发钥、重启恢复、命令行、浏览器演示、阶段计时、尺寸与内存统计 | 将密码原语转化为可操作、可测量的完整消息系统 |

**第一，建立面向异构格原语的 IB-ME 组合框架。** HIBE 和 IBS 使用彼此独立的参数对象、能力声明和版本号。新增后端只需实现相应接口、对象编码和自检逻辑，公共五算法、身份顺序和传输格式保持不变。十种常规组合以及 LATTE 与 Wang–PW 的扩展组合验证了该接口能够容纳多种格结构和对象规模。

**第二，实现 Sageloli 环格采样的结构化计算。**直接展开论文参数中的稠密矩阵会产生难以承受的内存和分解成本。本作品保留环多项式表示，并利用每个频率上的低秩形式计算所需平方根，使 PARAM-I 和 PARAM-II 均能完成系统建立、身份发钥、签名和验证。

**第三，实现 SRGG 对象的范围感知无损编码。** SRGG 公钥、主状态、用户密钥和签名中的字段具有不同取值范围。本作品分别采用模 $q$ 定长编码、带符号短整数编码和二元编码，

并在解码阶段检查长度与取值范围。三档主状态缩小约 5.05 至 5.46 倍，签名缩小约 2.55 至 2.65 倍。

**第四，实现固定身份对的 Lai 接收会话**。对于重复通信的接收者和发送者，Lai HIBE 的叶密钥可以在会话建立阶段生成并保存。后续每条消息仍独立完成认证和解密，但无需重复执行相同身份路径的委派。已有实验中，会话准备约 510 秒，准备完成后的解密中位数为 28.6 秒。

## 1.6 报告结构

第 2 章介绍 HIBE、IBS、IB-ME 通用构造以及本作品涉及的格困难问题。第 3 章给出公共五算法、混合加密、完整密文字节认证、拒绝路径和代表组合。第 4 章说明两种 HIBE、六类 IBS 的实现，并重点展开 Sageloli 结构化采样、SRGG 紧凑编码和 Lai 接收会话。第 5 章介绍软件架构、后端接入、状态管理、命令行与浏览器操作。第 6 章汇总功能、时间、尺寸、内存和恢复测试。第 7 章总结作品成果及后续工作。

# 第二章　基础知识与方案依据

身份基匹配加密并不是一种孤立的密码原语。它把分层身份基加密的访问控制能力与身份基签名的来源认证能力组合起来，再由统一协议处理消息容量、对象编码和完整性保护。本章先介绍这两类原语，再说明作品采用的格困难问题、具体算法和安全目标。第 3 章在此基础上给出完整协议。

## 2.1　分层身份基加密

传统公钥系统需要先取得用户公钥，再通过证书确认公钥属于谁。身份基密码把可读的身份字符串直接作为公开标识，例如邮箱、设备编号或组织内账号。可信发钥机构持有主秘密，核验用户身份后生成相应私钥。通信者只需知道对方身份和系统公开参数，不必为每个用户单独查询证书。

分层身份基加密（Hierarchical Identity-Based Encryption，HIBE）进一步把身份写成一条路径 $(\mathsf{id}_1, \ldots, \mathsf{id}_d)$。根机构可以为第一层身份生成密钥，上层用户还可以继续为下一层身份委派密钥。在本作品使用的两层结构中，第一层放接收者 $r$，第二层放发送者 $s$：

$$(\mathsf{id}_1, \mathsf{id}_2) = (r, s).$$

接收者取得一级密钥 $\mathsf{dk}_r$ 后，可以为自己认可的发送者 $s$ 生成叶密钥 $\mathsf{dk}_{r,s}$。加密者用公开参数和同一条身份路径产生密文。只有路径一致的叶密钥才能恢复其中的短秘密。

一套两层 HIBE 包含五项基本操作：

1. Setup 建立公开参数和主秘密；
2. Extract 为第一层身份生成接收密钥；
3. Derive 由接收密钥和第二层身份生成叶密钥；
4. Encrypt 沿两层身份路径加密；
5. Decrypt 使用叶密钥解密。

这里最重要的性质是身份路径绑定。即使两名接收者使用同一套系统参数，他们的一级密钥也不能互换；同一接收者为不同发送者得到的叶密钥同样不能互换。匿名 HIBE 还隐藏密文所对应的身份路径，这正是 Wang 通用构造用于保护通信关系的基础 [3]。

## 2.2 身份基签名

身份基签名（Identity-Based Signature，IBS）让验证者直接使用签名者身份和系统公开参数验签。发钥机构首先建立 IBS 公开参数 ipk 与主秘密 imsk，再为身份 $s$ 生成签名密钥：

$$(\mathsf{ipk}, \mathsf{imsk}) \leftarrow \mathsf{Setup}, \qquad \mathsf{sk}_s \leftarrow \mathsf{KeyExtract}(\mathsf{imsk}, s).$$

发送者用 $\mathsf{sk}_s$ 签署消息 $M$，接收者用 ipk、身份 $s$ 和消息验签。实现还提供签名加载方法，把传输字节恢复成严格类型的签名对象。

本作品接入了两种不同的 IBS 实现方式。第一种是原生格 IBS，身份直接进入格上的目标、挑战或陷门采样过程，Luo、SRGG、Sageloli 和 Wang–PW 属于这一类。第二种是认证转换：发钥机构为用户的普通签名公钥签发身份证书，用户再用对应私钥签消息；验证者依次检查机构证书和用户签名。本作品按照 BNN 通用转换实现后一种方式，并接入 ML-DSA 与 Falcon [9, 10, 18]。

两种方式在公共协议中的职责相同，都是把发送身份 $s$ 与待签字节绑定。区别主要体现在密钥生成、签名尺寸和运行资源。认证转换可以直接利用成熟的普通签名实现；原生 IBS 则能保留论文给出的身份结构和格关系。统一接口让这些差异停留在后端内部。

## 2.3 格结构与困难问题

格密码的安全性通常归结为在高维格中求解某类困难问题。本作品涉及的算法分布在普通矩阵格、环格、模格和 NTRU 结构格上。它们共享线性代数、误差采样和短向量等工具，但对象形状与可利用的代数结构不同。

### 2.3.1 LWE 与 SIS 问题

学习带误差问题（Learning With Errors，LWE）从带小误差的线性样本

$$(A,\ A\mathbf{s} + \mathbf{e} \bmod q)$$

中隐藏秘密 $\mathbf{s}$。误差 $\mathbf{e}$ 使线性方程不能直接求解。格加密通常利用这一结构隐藏明文或随机种子。短整数解问题（Short Integer Solution，SIS）则给出公开矩阵 $A$，要求找到非零短向量 $\mathbf{z}$ 使

$$A\mathbf{z} = \mathbf{0} \pmod q, \qquad \|\mathbf{z}\| \le \beta.$$

格签名常把消息哈希成一个目标向量，再用陷门采样一个短原像；伪造者若能产生两组适当短原像，便可导出 SIS 型短关系。

普通矩阵格直接保存 $A$ 及向量。环格把向量组织为商环 $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ 中的多项式，循环卷积取代稠密矩阵乘法；Ring-LWE 和 Ring-SIS 由此获得紧凑表示和快速变换。模格再把环元素组成小秩向量或矩阵，在普通格的灵活性与环格的结构性之间取折中；对应问题通常记为 Module-LWE 和 Module-SIS [12, 13]。

NTRU 结构也在多项式环上运算。短多项式关系既能构造陷门，也能支持快速签名和分层委派。LATTE 使用 NTRU 结构陷门与 Ring-LWE 密文路线；Falcon 则在 NTRU 格上构

造普通签名。作品在分类时保留各论文的具体假设与模型，不把“环”“模”简单当作安全等级标签。

### 2.3.2 困难问题与系统原语的对应关系

表 2.1把报告中的主要实现放在同一张图谱中。HIBE 决定谁能够恢复随机种子，IBS 决定恢复后的消息是否来自指定发送者。两边可以采用不同维度、模数和格结构，组合协议只要求它们满足统一的身份、容量和编码接口。

**表 2.1** 主要原语、格结构与系统职责

| 原语 | 主要格结构或困难问题 | 在作品中的对象 | 在 IB-ME 中的职责 |
|---|---|---|---|
| LATTE HIBE | NTRU 结构陷门、Ring-LWE 路线 | 两层接收密钥、叶密钥、32 字节种子胶囊 | 按 $(r, s)$ 路径封装种子 |
| Lai HIBE | 理想格陷门、PLWE 路线 | 逐层追加的身份块与委派陷门 | 按 $(r, s)$ 路径封装种子 |
| BNN + ML-DSA | Module-LWE / Module-SIS 及认证转换 | 机构证书、用户公钥与 ML-DSA 签名 | 认证发送身份与协议消息 |
| BNN + Falcon | NTRU 结构格签名及认证转换 | 机构证书、用户公钥与 Falcon 签名 | 认证发送身份与协议消息 |
| Luo | Module-LWE / Module-SIS | 模格公钥、响应和提示信息 | 原生身份签名 |
| SRGG | Module-LWE / Module-SIS | 身份目标、扰动承诺与短响应 | 原生身份签名 |
| Sageloli | Ring-SIS | 环陷门、身份盐、消息盐与短响应 | 原生身份签名 |
| Wang–PW | 矩阵 SIS | G 陷门、身份盐、消息盐与短原像 | 原生身份签名 |

这张对应表说明了“多类格困难假设”在系统中的具体位置：它不是同时把多种假设堆在一个数学公式中，而是让来自不同格结构的 HIBE 和 IBS 通过同一组合层工作。每套组合的密码性质仍由其两项底层原语共同决定。

## 2.4 通用构造的安全目标

Wang 等人的通用构造把两层匿名 HIBE 与 IBS 组合为 IB-ME [3]。本作品采用其中的普通隐私、加强认证与自我伪造防护目标。它们分别回答三个问题。

**消息和身份是否受到保护**。未持有匹配接收密钥的参与者不能恢复消息；匿名 HIBE 还使密文不直接暴露身份路径。第一层接收身份和第二层发送身份共同决定解密位置。

**发送来源是否可信**。接收者恢复载荷后，以自己期望的发送身份验签。攻击者不能把一份有效密文改写成另一发送者的消息，也不能只替换身份字段继续通过验证。

**合法用户能否冒充其他关系**。一个拥有自己密钥的用户也不应借此构造另一组发送、接收身份关系。HIBE 路径与 IBS 签名消息同时包含双方身份，使“谁接收”和“认可谁发送”落在同一次恢复过程中。

在软件中，这些目标被拆成三层实现。底层 HIBE 和 IBS 按照各自论文或标准运行；组合层固定身份顺序、签名输入和解密次序；传输层再对完整密文字节做一次性认证。三层各自解决不同问题，后两层不会修改底层格算法。

## 2.5 已实现算法与代表组合

日常组合矩阵由两种 HIBE 与五种 IBS 组成。两种 HIBE 是 LATTE 与 Lai；五种 IBS 是 BNN + ML-DSA、BNN + Falcon、Luo 模格、SRGG 和 Sageloli。十种配对均通过公共接口完成建立、发钥、加密、匹配解密和身份错配处理。Wang–PW 是随后完成的第六种 IBS，其目标规模实现已经与 LATTE 完成一套端到端组合；它的资源需求明显高于常规十组，因此单独列入第 4、6 章。

报告选取 A、B、C 三组代表展开说明：

- A 组为 LATTE + BNN/ML-DSA-44，展示标准化普通签名经认证转换接入后的低延迟基线；
- B 组为 LATTE + SRGG PARAM-I 紧凑格式，展示原生模格 IBS 及对象编码优化；
- C 组为 Lai C2 + Sageloli PARAM-I，展示理想格 HIBE、环格 IBS 和高成本委派场景。

三组不是简单的快慢排名。选择时同时考虑完整流程能否运行、格结构覆盖、在线时延、密钥和密文尺寸、峰值内存、恢复能力以及能否体现作品中的关键优化。A 适合高频消息基线，B 用于观察原生 IBS 和通信尺寸，C 用于研究高成本原语怎样通过结构化计算与会话复用进入完整系统。Wang–PW 则补充矩阵 SIS 目标规模路线。

参数说明采用各原语自己的论文或标准口径。ML-DSA-44 对应 NIST 安全类别 2；SRGG PARAM-I 论文给出的量子密钥恢复与伪造成本分别约为 95 bit 和 93 bit；Lai C2 的现有系数实例量子估计约 154.2 bit；Wang–PW 目标档按项目确定的约 115 bit 量子安全工作值使用。第 6 章比较性能时保留这些参数差异，不把不同原语的位数换算成一条统一排名。

## 2.6 本章小结

本章给出了作品的理论与算法坐标。两层 HIBE 沿 $(r, s)$ 路径控制随机种子的恢复，IBS 把发送身份与协议消息绑定；普通矩阵格、环格、模格和 NTRU 结构格分别进入不同后端。统一组合不要求两项原语共享参数，只要求它们遵守稳定接口。下一章将把这些原语组装成五个完整 IB-ME 算法，并说明混合加密、完整密文认证和错误处理的具体位置。

# 第三章　通用 IB-ME 系统设计

本章给出作品的公共协议。协议接收任意满足接口的两层 HIBE 和 IBS，把它们组成 Setup、发送者发钥、接收者发钥、Encrypt 和 Decrypt 五个算法。HIBE 始终只加密 32 字节随机种子，较大的签名和应用消息由对称加密承载；一次性签名负责保护完整密文字节。

## 3.1　参与者、对象与身份顺序

系统包含可信发钥机构、发送者和接收者。发钥机构保管 HIBE 与 IBS 的主秘密，公开两套公开参数。发送者领取与自身身份 $s$ 绑定的签名密钥，并在加密时指定接收者 $r$；接收者领取身份 $r$ 的 HIBE 一级密钥，并在解密时填写自己期望的发送者 $s$。

HIBE 路径固定为 $(r, s)$。这一顺序贯穿密钥、密文、状态文件和错误输入测试。接收者位于第一层，因此机构只需向 Bob 发放 $\mathsf{dk}_r$；Bob 在本地根据期望发送者派生 $\mathsf{dk}_{r,s}$。发送者不持有 HIBE 私钥，只使用公开参数加密。

表 3.1列出主要符号。两项底层原语分别维护参数和对象，公共协议不要求它们使用相同的 $q$、维度或环。

**表 3.1**　协议主要符号

| 符号 | 含义 |
|---|---|
| $r, s$；$r', s'$ | 密文指定的接收者与实际发送者；接收密钥身份与接收者期望的发送者 |
| $\mathcal{H}, \mathcal{S}$ | 两层匿名 HIBE 与 IBS |
| $\mathsf{hpk}, \mathsf{hmsk}$ | HIBE 公开参数与主秘密 |
| $\mathsf{ipk}, \mathsf{imsk}$ | IBS 公开参数与主秘密 |
| $\mathsf{dk}_r, \mathsf{sk}_s$ | 接收者一级密钥与发送者签名密钥 |
| $M, \sigma$ | 应用消息与 IBS 签名 |
| $z, K$ | 32 字节随机种子与由其派生的对称密钥 |
| $\mathsf{ovk}, \mathsf{osk}, \tau$ | 一次性验签公钥、一次性签名私钥与外层签名 |
| $\mathrm{LP}(x)$ | 4 字节长度前缀与字节串 $x$ 的拼接 |

公共对象由两项原语对象和协议元数据组成。系统公开对象包含 $(\mathsf{hpk}, \mathsf{ipk}, \mathsf{suite})$，其中 suite 记录算法、版本、参数快照和编码格式。主状态保存 $(\mathsf{hmsk}, \mathsf{imsk})$ 及发钥账本。发送密钥记录身份 $s$ 和 $\mathsf{sk}_s$；接收密钥记录身份 $r$ 和 $\mathsf{dk}_r$。每个对象都绑定同一套组合配置。

## 3.2 IB-ME 五算法设计

### 3.2.1 系统建立算法 Setup

**输入：** 组合配置 suite、随机源。 **输出：** 系统公开对象 MPK 和主状态 MSK。

建立过程先检查 HIBE 与 IBS 的参数、身份容量、消息容量和编码格式。检查通过后分别运行

$$(\mathsf{hpk}, \mathsf{hmsk}) \leftarrow \mathcal{H}.\mathsf{Setup}, \qquad (\mathsf{ipk}, \mathsf{imsk}) \leftarrow \mathcal{S}.\mathsf{Setup}.$$

随后计算两套公开参数和 suite 的规范摘要，写入所有上层对象。公开对象可分发给通信双方；主状态仅由发钥机构保存。

### 3.2.2 发送者密钥生成算法 SenderKeyGen

**输入：** MSK、发送身份 $s$。 **输出：** 发送密钥 $\mathsf{EK}_s$。

机构先将外部身份 $s$ 编码为 IBS 接受的规范字节，再调用

$$\mathsf{sk}_s \leftarrow \mathcal{S}.\mathsf{KeyExtract}(\mathsf{imsk}, s).$$

得到的 $\mathsf{EK}_s = (s, \mathsf{sk}_s, \mathsf{suite})$ 交给发送者。固定发钥服务会保存已发身份及对象摘要；同一身份再次申请时读取既有密钥，不改变该身份的签名状态。

### 3.2.3 接收者密钥生成算法 ReceiverKeyGen

**输入：** MSK、接收身份 $r$。 **输出：** 接收密钥 $\mathsf{DK}_r$。

机构把 $r$ 编码为 HIBE 第一层身份并调用

$$\mathsf{dk}_r \leftarrow \mathcal{H}.\mathsf{Extract}(\mathsf{hmsk}, r).$$

接收密钥只包含第一层秘密。期望发送者 $s$ 在解密时才进入 $\mathcal{H}.\mathsf{Derive}(\mathsf{dk}_r, s)$，因此 Bob 可以用一份机构发放的密钥接收来自多个指定发送者的消息。

### 3.2.4 消息加密算法 Encrypt

**输入：** MPK、发送密钥 $\mathsf{EK}_s$、接收身份 $r$、消息 $M$。 **输出：** 密文 $C$。

加密过程由内向外建立密文：

1. 生成一次性签名密钥对 $(\mathsf{ovk}, \mathsf{osk})$，计算验签公钥承诺 $d_{\mathrm{ovk}}$；
2. 将 $r$、$M$ 与 $d_{\mathrm{ovk}}$ 编码成 IBS 消息 $E$，用 $\mathsf{sk}_s$ 产生签名 $\sigma$；
3. 把 $E$ 和 $\sigma$ 编码成内部载荷 $P$；
4. 新取 32 字节种子 $z$，沿 HIBE 路径 $(r, s)$ 加密得到胶囊 $C_0$；
5. 由 $z$ 派生对称密钥 $K$，加密 $P$ 得到 $C_1$；
6. 对公开上下文、ovk 以及完整内层字节 $B = (C_0, C_1)$ 签名，得到 $\tau$；

7. 规范编码 $(\mathsf{ctx}, \mathsf{ovk}, B, \tau)$，输出 $C$。

每条消息使用新的 $z$、GCM nonce 和一次性签名密钥。发送者的长期 IBS 密钥只负责身份认证，一次性密钥只负责当前密文字节。

### 3.2.5 匹配解密算法 Decrypt

**输入：** MPK、接收密钥 $\mathsf{DK}_{r'}$、期望发送身份 $s'$、密文 $C$。 **输出：** 消息 $M$ 或拒绝符号 $\perp$。

接收过程与加密顺序相反：

1. 解析外层格式，检查版本、字段长度、组合配置和公开上下文；
2. 用 ovk 验证 $\tau$，确认内层字节在传输中没有被替换、截断或追加；
3. 调用 $\mathcal{H}.\mathsf{Derive}(\mathsf{dk}_{r'}, s')$ 得到叶密钥；
4. 解开 $C_0$ 得到 $z$，派生 $K$ 并验证 GCM 标签，恢复载荷 $P$；
5. 解析 $E$ 与 $\sigma$，以身份 $s'$ 验证 IBS 签名；
6. 核对 $E$ 中的接收身份是 $r'$，并核对其中的 $d_{\mathrm{ovk}}$ 来自外层 ovk；
7. 所有检查通过后返回 $M$。

该顺序先处理成本较低的格式与外层认证，再进入 HIBE 委派和解密。身份错配最终会在 HIBE 路径、IBS 签名或内部身份核对处被发现，统一返回 $\perp$。

## 3.3 基于短种子封装的混合加密

IBS 签名的尺寸差异很大。认证转换签名通常为数 KiB，Sageloli 和 Wang–PW 签名达到数百 KiB 或数 MiB，而 HIBE 的直接明文容量通常只有几十字节。通用协议因此采用 KEM–DEM 式混合结构：HIBE 只加密固定的随机种子，对称加密承担可变长度载荷。

外部身份先用角色标识、长度前缀和零填充编码为定长输入。当前 64 字节外部身份对应 86 字节内部身份；接收者与发送者使用不同角色字节。HIBE 封装和密钥派生为

$$
\begin{aligned}
C_0 &\leftarrow \mathcal{H}.\mathsf{Encrypt}(\mathsf{hpk}, (\mathrm{Pad}_P(r), \mathrm{Pad}_C(s)), z),\\
K &\leftarrow \text{HKDF-SHA256}(z; \mathsf{ctx}_H, \mathsf{D}_{\mathrm{kdf}}),\\
C_1 &\leftarrow \text{AES-256-GCM.}\,\mathrm{Enc}(K, n, P; \mathsf{AAD}).
\end{aligned}
$$

这里 $n$ 是每条消息新生成的 12 字节 nonce。HKDF 和 GCM 分别采用 RFC 5869 与 NIST SP 800-38D 规定的原语 [14, 15]。附加认证数据包含组合上下文、HIBE 胶囊 $C_0$、nonce 和载荷长度。这样，修改 HIBE 胶囊或把对称密文移动到另一套组合中都会导致 GCM 验证失败。

混合结构把“底层 HIBE 可以加密多少字节”固定为 32 字节。应用容量转而由 IBS 的签名输入上限、签名最大尺寸和所选载荷预算共同决定。新增后端时，装配层可在建立主密钥之前完成容量计算。

## 3.4 完整密文字节认证

只验证解密后的 IBS 签名还不够。传输中的版本、字段边界、HIBE 胶囊和 GCM 密文也需要统一保护。作品采用 Lamport 型一次性签名 [16] 连接内外两层认证。

加密时先计算一次性公钥承诺，并把它与应用消息一起交给 IBS：

$$
\begin{aligned}
d_{\mathrm{ovk}} &= H_{\mathrm{commit}}(\mathsf{ovk}),\\
E &= \mathsf{D}_{\mathrm{bound}} \parallel \mathrm{LP}(d_{\mathrm{ovk}}) \parallel \mathrm{LP}(r) \parallel \mathrm{LP}(M),\\
\sigma &\leftarrow \mathcal{S}.\mathsf{Sign}(\mathsf{sk}_s, \mathrm{EncMsg}(r, E)),\\
P &= \mathsf{D}_{\mathrm{sig}} \parallel \mathrm{LP}(\sigma) \parallel \mathrm{LP}(E).
\end{aligned}
$$

混合加密得到内层字节 $B$ 后，再计算

$$
\tau \leftarrow \mathsf{OTS.Sign}_{\mathsf{osk}}(\mathsf{D}_{\mathrm{ct}} \parallel \mathrm{LP}(\mathsf{ctx}) \parallel \mathrm{LP}(\mathsf{ovk}) \parallel \mathrm{LP}(B)).
$$

接收者先用 ovk 验证外层签名，再解密 $B$ 并检查 IBS 签名中的 $d_{\mathrm{ovk}}$。攻击者即使重新生成一组一次性密钥并对修改后的 $B$ 签名，也无法让内部 IBS 签名承诺新的公钥。反过来，原一次性签名也无法验证任何变化后的内层字节。

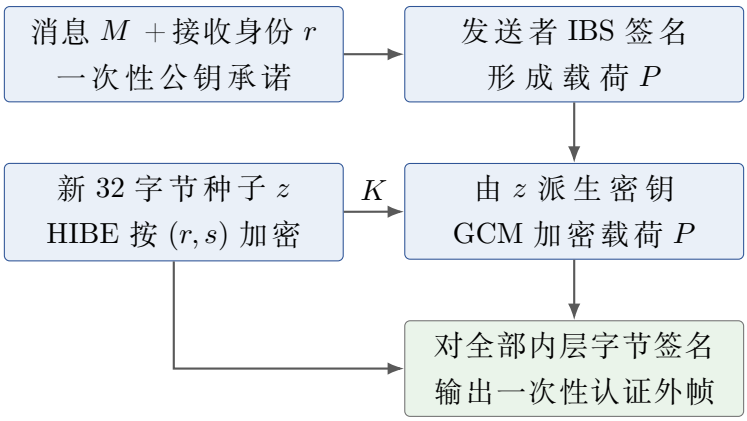



**图 3.1** 身份签名、HIBE 种子封装与完整字节认证的关系

## 3.5 协议正确性与异常处理

当 $r = r'$ 且 $s = s'$ 时，接收端派生的叶密钥与加密路径相同，因此 HIBE 解密恢复同一个 $z$；HKDF 得到同一个 $K$，GCM 恢复 $P$；IBS 验证使用同一发送身份 $s$ 和接收身份 $r$，内外一次性公钥承诺也一致，最终输出 $M$。

当接收身份错误时，一级密钥无法形成正确叶密钥，HIBE 种子恢复失败。当期望发送身份错误时，第二层路径和 IBS 验签身份都会改变。当应用消息、IBS 签名或一次性公钥承诺变化时，IBS 验证失败；当胶囊、对称密文、上下文、nonce 或外层签名变化时，外层签名或 GCM 标签失败。

格式解析采用固定魔数、版本、域标识和长度前缀。解码器先检查总长度和字段上限，再分配对象；未知字段、重复字段、截断、追加、非规范整数、错误对象类型和配置摘要不匹配

均直接拒绝。执行环境缺少依赖或超过资源上限时，程序返回明确运行错误，不把它解释为一次身份错配。

## 3.6 代表组合与应用消息容量

表 3.2列出 A、B、C 三组代表。三组共享本章协议，区别只在 HIBE、IBS 和对应参数。A 用于观察标准化认证转换的低延迟完整流程；B 用于分析原生模格 IBS 与紧凑编码；C 用于分析 Lai 委派与 Sageloli 结构化采样。

**表 3.2** 三组代表及应用容量

| 组 | HIBE | IBS | 消息上限/B | 签名预算/B |
|---|---|---|---|---|
| A | LATTE-3 | BNN/ML-DSA-44 | 59,191 | 6,237 |
| B | LATTE-3 | SRGG PARAM-I 紧凑格式 | 65,364 | 15,704 |
| C | Lai C2 | Sageloli PARAM-I | 3,924 | 3,227,824 |

A、B 的混合载荷预算分别为 64 KiB 和 128 KiB。C 的签名约 3.08 MiB，采用 8 MiB 预算；其 3,924 字节应用上限由 IBS 的 4,096 字节输入限制扣除协议编码得到。三组 HIBE 输入始终是 32 字节种子，扩大对称载荷预算不会改变底层 HIBE 明文。

代表选择综合考虑五项因素：完整协议是否已经运行；HIBE 与 IBS 所覆盖的格结构；在线加解密时延；密钥、签名、密文和内存；以及能否体现本作品的优化方法。三组分别承担不同观察任务，第 6 章按这些维度分析实验结果。

## 3.7 本章小结

公共协议把底层原语的差异收敛到稳定的五个 IB-ME 算法。32 字节种子封装解决 HIBE 与大签名之间的容量矛盾，IBS 签名绑定发送者、接收者、消息和一次性公钥承诺，外层一次性签名保护完整传输字节。下一章转入各底层原语和三项工程优化，说明这些公共步骤具体调用了哪些格算法。

# 第四章　关键原语实现与工程优化

公共协议只看到 HIBE 与 IBS 接口，实际计算则由各后端完成。本章先说明两种 HIBE 和主要 IBS 的实现，再展开结构化采样、紧凑编码和接收会话三项优化。每一节都从该原语在 IB-ME 中的职责出发，随后给出关键方程和软件中的对象表示。

## 4.1　两种分层身份基加密

### 4.1.1　LATTE 两层身份委派机制

LATTE 在负循环环 $R_q = \mathbb{Z}_q[x]/(x^N+1)$ 上建立 NTRU 结构陷门 [4]。作品采用 LATTE-3 参数：$N = 1024$、$q = 68718428161$，分层深度为 2。建立阶段生成短多项式 $f, g$，求解 NTRU 方程

$$fG - gF = q, \qquad h = gf^{-1} \pmod q, \qquad S_0 = \begin{pmatrix} g & -f \\ G & -F \end{pmatrix}.$$

$h$ 进入公开参数，$S_0$ 作为根陷门保存在主状态中。

接收身份 $r$ 经过独立域标识的哈希得到 $A_1 = H_1(r)$。发钥算法利用根陷门为第一层身份生成短基，得到接收密钥 $\mathsf{dk}_r$。接收者指定发送身份 $s$ 后，再计算 $A_2' = H_2(r, s)$ 并委派叶钥 $(t_0, t_1, t_2)$。叶钥满足

$$t_0 + t_1 h + t_2 A_1 = A_2' \pmod q.$$

这条关系既用于解密，也是密钥从磁盘恢复时的绑定检查。若把另一个接收身份或发送身份放入同一对象，等式右端随之变化，加载器会拒绝该组合。

加密算法取短噪声，把 32 字节种子编码进环元素，并沿 $(r, s)$ 路径生成胶囊。解密使用叶钥消去公开环项，恢复种子后再执行一次确定性重加密核对。该核对用于排除落在解码区间内的错误结果，随后才把种子交给混合加密层。

实现复用了 LATTE 作者代码中的 Gaussian 采样和 ffSampling 路径，并把 C 对象包装为严格的 Python 后端。接入过程中修正了 COSAC 读取 Box–Muller 新缓冲的位置；每次原生采样完成后，Python 层使用精确环算术检查 NTRU 关系、委派关系和对象形状。公开参数、主陷门、一级密钥、叶密钥和密文分别使用独立对象类型，参数快照随对象保存。

LATTE 的建立和一级发钥时间在三组代表中都较低，因此 A、B 共用它作为 HIBE。相同 HIBE 也使两组差异主要来自 IBS，便于分析认证转换与原生模格签名在完整协议中的开销。

### 4.1.2　Lai 分层身份块扩展机制

Lai 方案使用理想格陷门组织分层委派 [5]。根公开向量与短陷门满足

$$\mathbf{a}_0^{\mathsf{T}} R + \mathbf{a}_1 = \mathbf{g}.$$

两层身份分别映射为二进制环元素 $\mathrm{id}_1$ 和 $\mathrm{id}_2$。第一层公开项与完整身份公开项写成

$$\mathbf{a}_{\mathrm{id}_1} = \mathbf{u}_0 + \mathrm{id}_1\mathbf{u}_1, \qquad \mathbf{a}_{\mathrm{ID}} = \mathbf{a}_{\mathrm{id}_1} + \mathrm{id}_2\mathbf{u}_2.$$

发钥和委派每进行一层，就向公开矩阵追加对应身份块，并为扩展后的矩阵生成新陷门。接收者 $r$ 对应第一层，期望发送者 $s$ 对应第二层。

加密时取短矩阵 $R_0, R_1, R_2$，组合为

$$R_{\mathrm{ID}} = R_0 + \mathrm{id}_1 R_1 + \mathrm{id}_2 R_2, \qquad \mathbf{e}_1 = -\mathbf{e}_0^{\mathsf{T}} R_{\mathrm{ID}},$$

再计算

$$\mathbf{u} = F_1\mathbf{s} + p\mathbf{e}_0, \qquad \mathbf{v} = (\mathbf{a}_{\mathrm{ID}} + h'\mathbf{g})\mathbf{s} + p\mathbf{e}_1 + \mathbf{m}.$$

叶陷门对 $(\mathbf{u}, \mathbf{v})$ 执行 Babai 反演，检查误差的模 $p$ 结构并恢复 32 字节种子。

C 组采用 $n = 2048$、$q = 274877976577$、$k = 39$、深度 2、$p = 2$ 的 C2 配置，噪声参数为 $\sigma = 2$、$\sigma_r = 1$。身份映射使用二进制系数，避免身份乘短矩阵时放大噪声。实现按层追加身份块，统一矩阵形状，并用 Klein/GPV 路径完成陪集采样。公开参数、根陷门、两级密钥与密文均按 5 字节系数编码。

Lai 的主要成本在二级委派。一次完整 C 组解密中，委派占总时间的绝大部分；第 4.5 节因此把固定身份对的委派移动到接收会话准备阶段。

## 4.2 身份基签名后端

### 4.2.1 BNN 认证转换与 ML-DSA / Falcon 接入

BNN 转换把普通数字签名变为身份基签名 [9]。系统建立时，发钥机构生成一套机构签名密钥。为身份 $s$ 发钥时，先为用户生成普通签名密钥对 $(\mathsf{vk}_s, \mathsf{xk}_s)$，再由机构签署

$$\mathsf{cert}_s = \mathsf{DS.Sign}_{\mathsf{sk}_{\mathrm{CA}}}(\mathsf{Enc}(s, \mathsf{vk}_s)).$$

用户签名密钥由 $\mathsf{xk}_s$、$\mathsf{vk}_s$ 和 $\mathsf{cert}_s$ 组成。签消息时，用户对上下文、身份、公钥、证书和协议消息的规范编码签名。完整 IBS 签名携带用户公钥、证书和用户签名；验证者先检查证书，再检查用户签名。

同一转换核心接入 ML-DSA 与 Falcon 两个普通签名引擎。ML-DSA 实现固定 FIPS 204 的参数与对象格式 [10]；A 组选用 ML-DSA-44，其基础公钥为 1,312 字节、基础签名为 2,420 字节，组合后的 IBS 签名为 6,237 字节。Falcon 后端采用 Falcon-512 和 Falcon-1024 的固定作者实现与 padded 格式 [18]。两者共享机构证书流程，但底层密钥、签名编码和版本号完全分开。

认证转换的优势是建立和在线签名较快，也便于复用标准化实现。其额外空间来自用户公钥、机构证书和用户签名三部分。A 组的实验说明，即使采用较小的 ML-DSA 参数，完整密文仍会受到一次性认证和 HIBE 胶囊的固定开销影响。

### 4.2.2 Luo 模格身份签名

Luo 后端实现论文的模块格 IBS [17]。身份哈希、消息挑战、响应和提示信息直接进入签名方程，验证者只需公开参数、身份和消息即可验签。实现保留参考格式，同时提供无损紧凑格式；两种格式加载后得到同一数学对象，签名算法和验签方程不变。

Luo 后端在组合矩阵中分别与 LATTE、Lai 配对。它不是 A、B、C 三组的重点代表，但承担两个作用：一是验证公共协议能够接入另一种原生模格 IBS；二是为十组同机补测提供与 SRGG 不同的模格实现。后端公开身份上限、签名输入上限和签名尺寸，装配层据此计算应用容量。

### 4.2.3 SRGG 身份目标与消息挑战关系

SRGG 采用模块格结构，公开矩阵写成

$$A = [I_d \mid \widehat{A} \mid B]$$

[6]。身份 $s$ 决定目标 $h = H_1(P, s)$，用户短见证 $y$ 满足 $Ay = h \pmod{Q}$。签名构造矩阵和响应为

$$M_h = [\,2A \mid Qj - 2h\,] \pmod{2Q}, \qquad z = p_0 + \begin{bmatrix} y \\ 1 \end{bmatrix} (\zeta u + c),$$

其中 $\zeta = 1 + x^{n/2}$、$j = \zeta^\star e_1$。扰动 $p_0$ 的承诺和消息共同决定挑战 $c$。验证者由

$$W' = M_h z - Qjc$$

恢复承诺，重新计算挑战，并检查响应范数。

B 组采用 PARAM-I：多项式次数 $n = 512$、模块秩 2、$p = 16383$、$Q = 15744063$，gadget 基为 31、位数为 2。建立阶段生成公开矩阵和短陷门；身份发钥执行相关扰动与短原像采样；签名阶段重新采样承诺和挑战。实现同时保留 JSON 参考格式与紧凑格式，二者由同一后端运算。第 4.4节说明四类对象的位级编码。

### 4.2.4 Sageloli 环陷门与非齐次签名方程

Sageloli 的公开向量与主陷门 $R$ 满足 [7]

$$A = [\bar{A} \mid g - \bar{A}R].$$

为身份 $s$ 发钥时，算法先取身份盐 $\rho$，计算 $h = H_1(P, \rho, s)$，再采样身份陷门 $T_s$，使

$$AT_s = h - g \pmod{q}.$$

签名另取消息盐 $\eta$，通过相关扰动和三进制陪集采样得到响应 $z$。验证者检查范数及

$$[A, h]z = H_2(P, \rho, s, \eta, M') \pmod{q}.$$

C 组采用 PARAM-I：$(d, k, \ell) = (2048, 65, 132)$、$q = 3^{65}$、$r^2 = 49$；身份与签名采样宽度平方分别为 2720281532542 和 270116709797628319132317。PARAM-II 保持同一算法而增大参数。两档均使用环原生后端、规范对象和固定身份发钥服务。

该方案的难点不是验签方程，而是论文参数下的相关扰动。若把环乘法展开为普通矩阵，单个中间矩阵就超过常规工作站内存。下一节说明作品如何保留环结构完成采样。

### 4.2.5 Wang–PW 目标规模矩阵 SIS 实现

这里的 Wang–PW 指 Wang 基本 IBS 加 Pan–Wagner 自适应转换，它与第 2 章所述的 Wang 通用 IB-ME 构造是两个层次：前者是可替换的 IBS 后端，后者规定 HIBE 与 IBS 怎样组合。基本 IBS 采用矩阵 SIS 路线，Pan–Wagner 转换为身份和消息分别加入随机盐 [3, 8]。

目标档参数为

$$n = 512, \qquad q = 4294967291, \qquad s_0 = 4.$$

令 $k = \lceil \log_2 q \rceil = 32$，则 gadget 宽度 $w = nk = 16384$。实现取 $\bar{m} = w$、$m = \bar{m} + w = 32768$，签名向量维数为 $M = m + w = 49152$。根公开矩阵由 G 陷门构造：

$$A = [\bar{A} \mid G - \bar{A}R],$$

其中短矩阵 $R$ 只保存在发行机构的主状态中。

发钥时，Pan–Wagner 外层先为身份 $s$ 生成固定身份盐 $\rho$，再计算 $H_1(P, \rho, s)$。后端逐列采样用户陷门 $X$，使

$$AX + H_1 = G, \qquad F_s = [A \mid H_1], \qquad F_s \begin{bmatrix} X \\ I \end{bmatrix} = G \pmod{q}.$$

用户密钥只保存 $X$、身份盐和公开上下文，不包含主陷门 $R$。签名时重新生成消息盐 $\eta$，计算消息目标 $H_2(P, \rho, s, \eta, M')$，再使用用户 G 陷门采样短向量 $z$：

$$F_s z = H_2(P, \rho, s, \eta, M') \pmod{q}.$$

完整签名为 $(\rho, \eta, z)$。验签重新计算 $H_1$ 和 $H_2$，检查维数、非零条件、整数平方范数和模方程。

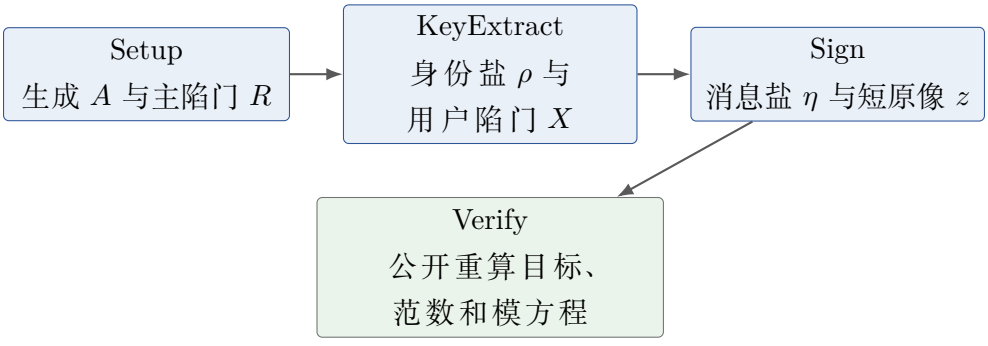



**图 4.1** Wang–PW 后端的身份发钥、签名与公开验签

有限离散 Gaussian 由独立 C++ 组件执行。实现使用 256 bit MPFR 精度、192 bit 概率质量和 20 个标准差截断窗口；根采样、用户发钥和签名分别使用固定宽度与公开门限。Python 层负责参数快照、域分隔哈希、对象编码和精确模方程，原生组件只承担有限 Gaussian 采样。

目标档的公钥约 64.0 MiB，主状态约 576 MiB，用户签名密钥约 2.06 GiB，签名约 384 KiB。大对象采用流式保存和恢复，签名进程只读取用户密钥与公开参数。固定发行服务记录身份盐和已发布密钥，使同一身份在重启后继续使用同一用户对象。

该实现已完成独立签名、公开验签、错误身份与消息、签名篡改、状态恢复以及 LATTE 组合。项目按约 115 bit 量子安全工作值使用该目标档。它的 Setup、发钥和签名消耗显著高于常规十组，第 6 章单独列出分阶段实测结果。

## 4.3 Sageloli 结构化环采样优化

### 4.3.1 稠密矩阵展开的资源瓶颈

Sageloli 的发钥和签名都需要在格陪集中采样。相关扰动由陷门决定其形状，再与离散采样配合，得到满足方程和范数要求的短向量。实现既要得到正确的相关性，也要承受论文参数下的对象规模。

环元素是一组多项式系数，乘法可以直接在负循环环上计算。如果把每个环元素展开成旋转矩阵，后续陷门与协方差也会随之变成很大的实数矩阵。PARAM-I 取 $d = 2048$、$\ell = 132$、$k = 65$，签名相关扰动的系数维数为

$$D = d(\ell + k) = 403456.$$

仅显式保存一个 $D \times D$ 实数矩阵，按每项 8 字节估算就需要约 1.18 TiB，还未计入分解工作区。因此，实际实现需要利用原式的结构来组织计算。

本作品始终保存多项式系数。公钥、主陷门和身份陷门分别以 $\ell \times d$、$(\ell - k) \times k \times d$ 和 $\ell \times k \times d$ 的系数数组表示。环乘法在原环上执行，结果仍检查

$$A = [\bar{A} \mid g - \bar{A}R], \qquad AT_s = H_1(P, \rho, s) - g \pmod q.$$

这样，公钥和密钥关系保持不变，计算过程避免展开旋转矩阵。

### 4.3.2 逐频率低秩分解方法

设当前陷门为 $T$，其形状为 $\ell \times k$ 个环元素，令

$$J = \begin{bmatrix} -T \\ I_k \end{bmatrix}, \qquad a = s^2 - r^2, \qquad c = 9r^2.$$

这里 $r$ 是基础采样宽度，$s$ 是该次相关扰动使用的宽度。需要计算的协方差形状为

$$\Sigma = aI - cJJ^\star,$$

其中 $\star$ 表示环上的伴随。这个式子的主要部分是标量矩阵 $aI$，修正部分由只有 $k$ 列的 $J$ 产生。利用这一点，可以将分解限制在较小的子空间。

第一步，对多项式执行负循环 Fourier 变换。在每个频率 $\nu$ 上，环乘法变为复数乘法，$J$ 对应一个 $(\ell + k) \times k$ 的复数矩阵 $J_\nu$。第二步，计算 Gram 矩阵

$$G_\nu = J_\nu^* J_\nu = U_\nu \operatorname{diag}(\lambda_i) U_\nu^*.$$

$G_\nu$ 只有 $k \times k$；在 PARAM-I 中，最大特征分解为 $65 \times 65$。

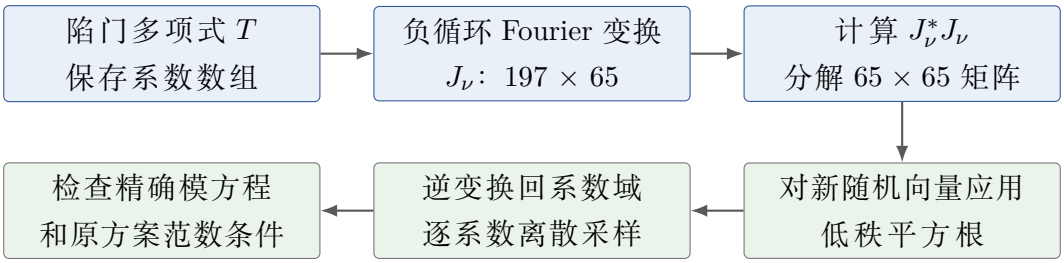



**图 4.2** 结构化采样的计算步骤；矩阵形状对应 PARAM-I

第三步，利用小矩阵的特征分解构造平方根算子

$$L_\nu = \sqrt{a} I - J_\nu U_\nu \operatorname{diag}\left( \frac{c}{\sqrt{a} + \sqrt{a - c\lambda_i}} \right) U_\nu^* J_\nu^*.$$

该式满足 $L_\nu L_\nu^* = aI - cJ_\nu J_\nu^*$。对新随机向量应用 $L_\nu$ 时，按右侧的小矩阵与向量乘法逐步计算，无需形成完整的 $L_\nu$。随后逆变换回系数域，继续执行原方案要求的离散 Gaussian 和三进制陪集采样。方法使用格陷门采样中的扰动思想 [11]，并针对本方案的环结构组织执行。

需要保存的主要数据量为 $O(d\ell k + dk^2)$，对应陷门系数和各频率的小矩阵计划。相较 $O(D^2)$ 的稠密表示，这直接降低了表示和分解规模。已经计算好的计划可在同一陷门下复用，每次发钥和签名仍生成新的随机样本。

### 4.3.3 数值采样与精确代数校验

Fourier 步骤使用数值运算，方案中的模方程则必须精确成立。实现将两部分分开：数值计划在建立时检查正定充分条件和变换误差；回到系数域后，环方程由整数运算检查。大模数 $q = 3^k$ 的公开乘积使用 FLINT 整数多项式，私有小整数乘法采用双素数 NTT 与 CRT 恢复，并检查整数卷积幅度落在可唯一恢复的范围内。

签名生成后继续执行原方案的范数门与验签方程。这样，浮点计算的职责是产生相关扰动，精确算术的职责是验证代数关系；两者在采样接口处衔接。第 6 章给出 PARAM-I 与 PARAM-II 的实际运行成本。

## 4.4 SRGG 对象的范围感知编码

### 4.4.1 参考表示的空间开销

SRGG 的公钥、陷门和签名含有大量多项式系数。JSON 将每个整数写成十进制字符串，还需要负号、逗号、括号和字段名。对大对象，表示开销会累积到与数学数据相近甚至更大的规模。

紧凑编码依据字段的数学取值范围选择位数。以取值为 $-1, 0, 1$ 的陷门系数为例，三种可能值只需 2 bit；公开模 $Q = 15744063$ 的系数落在 $[0, Q)$，使用 24 bit。对短密钥和签名响应，先以 Zigzag 映射处理正负号，再选取能够容纳该字段的最小统一位宽。

**表 4.1** SRGG 不同字段的编码方法

| 数据 | 存储方法 | 解码时检查 |
|---|---|---|
| 公开模 $Q$ 系数 | 每系数 24 bit | $0 \le x < Q$ |
| 三值陷门系数 | 每系数 2 bit | 拒绝第四个未使用码字 |
| 短密钥、响应整数 | Zigzag 与最小统一位宽 | 数值范围、位宽最小性 |
| 二元挑战 | 每系数 1 bit | 固定多项式次数与长度 |
| 类型、参数与上下文 | 固定头部和长度前缀 | 对象类别、公钥、参数一致 |

对含 $N$ 个系数、统一位宽为 $b$ 的字段，其主要编码长度为 $\lceil Nb/8 \rceil$ 字节，再加必要头部。解码后得到与 JSON 格式相同的数学对象，签名和验证继续使用原算法。

### 4.4.2 四类对象的规范编码与解码

该方法同时用于公钥、主状态、用户密钥和签名。各类对象有独立类型标识，系数顺序固定，末字节未使用位必须为零。解析器先根据形状计算长度，再读取数据，并检查参数、身份、公钥上下文及取值范围。解码后重新编码应得到相同字节串，从而保证同一个对象只有一种规范表示。

对象尺寸的改善会减少密钥文件和传输字节，实际收益取决于该对象在完整协议中的占比。例如，签名变短可以直接缩短对称加密载荷，但 HIBE 胶囊和一次性认证仍占固定空间。第 6 章将四类对象的编码对照与完整密文组成放在一起分析。编解码也有计算成本，实验同时保留该项测量。

## 4.5 Lai 接收会话优化

### 4.5.1 重复委派的性能瓶颈

在 Lai 组合中，接收者持有第一层密钥 $\mathsf{dk}_r$。解密来自发送者 $s$ 的消息前，需要运行

$$\mathsf{dk}_{r,s} \leftarrow \mathsf{Derive}(\mathsf{dk}_r, s).$$

同一接收者连续读取同一发送者的消息时，委派输入保持相同；反复执行会重复支付生成叶钥的开销。加密种子、密文与每条消息的签名则各不相同，需要逐条处理。

本作品引入显式接收会话：在会话建立时生成并验证一次叶钥，后续在固定双方身份下复用。图 4.3展示计算安排的变化。

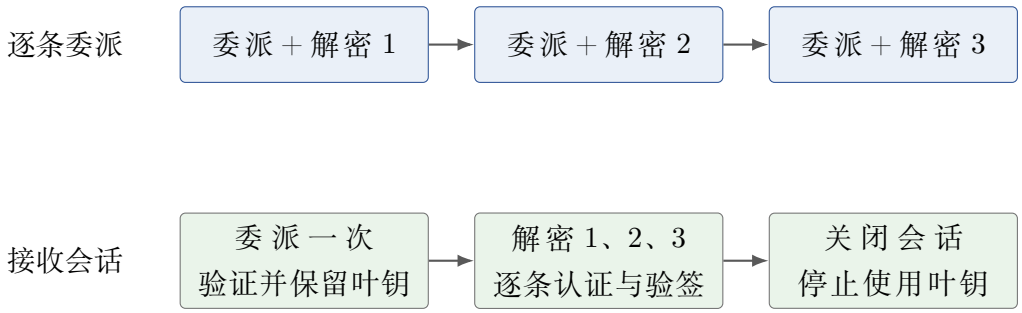



**图 4.3** 会话把与密文无关的委派安排到准备阶段

### 4.5.2 身份绑定与会话生命周期

会话建立时检查公开上下文、接收者身份和一级钥绑定；生成叶钥后检查完整 $(r, s)$ 路径及陷门方程。会话只服务于该公钥和身份对，关闭后停止解密。每条消息继续执行规范解析、外层字节认证、HIBE 解封装、GCM 验证、IBS 验签及一次性公钥承诺核对。

叶钥作为秘密状态保留在进程内，使用者显式关闭会话，会话对象不支持序列化。长期保存的接收密钥仍采用原持久格式。该设计适合双方身份固定的连续通信，并带来明确的额外密钥存储成本。

### 4.5.3 连续消息的摊销成本

设一次会话准备耗时为 $T_{\mathrm{prep}}$，会话内单条解密为 $T_{\mathrm{sess}}$。处理 $N$ 条消息的总成本与平均成本分别为

$$T_{\mathrm{total}}(N) = T_{\mathrm{prep}} + NT_{\mathrm{sess}}, \qquad \bar{T}(N) = \frac{T_{\mathrm{prep}}}{N} + T_{\mathrm{sess}}.$$

首条消息承担全部准备成本；连续消息逐渐摊薄这部分开销。第 6 章同时展示准备、后续解密和保留叶钥大小，以判断实际使用方式。

## 4.6 本章小结

本章说明了公共接口背后的具体计算。LATTE 通过 NTRU 结构陷门完成两层委派，Lai 通过追加身份块形成叶陷门；BNN 转换、Luo、SRGG、Sageloli 和 Wang–PW 分别提供不同结构的身份签名。六种 IBS 在公共协议中的职责相同，但签名生成、对象尺寸和运行资源差异明显。

三项优化直接对应这些差异。Sageloli 利用环结构把大分解转成逐频率低秩计算，SRGG 按字段范围编码四类对象，Lai 会话复用固定身份对的叶密钥。第 6 章将用实际时间、尺寸和内存说明每项方法带来的变化。

# 第五章　软件系统与操作流程

密码原语解决具体数学问题，软件系统还要负责组合选择、参数检查、对象保存和用户操作。本章介绍系统怎样把后端、公共协议和应用入口分开，以及新增 HIBE 或 IBS 时需要完成哪些工作。

## 5.1　分层软件架构

系统自下而上分为基础设施层、密码原语层、组合层、协议层和应用层。基础设施层提供规范编码、随机源、域分隔哈希、错误类型和原子文件保存。密码原语层分别实现 HIBE 和 IBS。组合层根据配置选择后端，并在生成密钥前完成能力与容量检查。协议层实现第 3 章的五个 IB-ME 算法。最上层提供命令行、实验工具和浏览器演示。

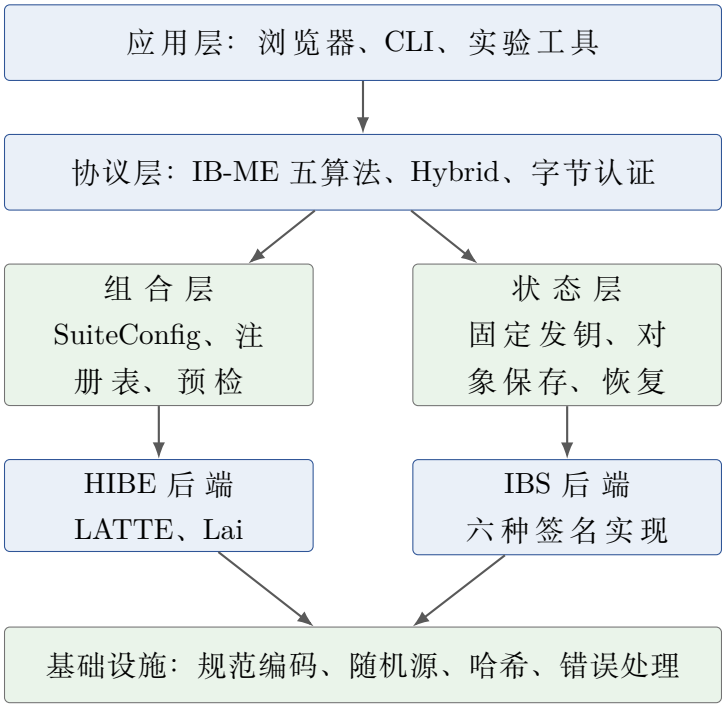



**图 5.1**　软件系统的分层结构

这一结构有两个直接作用。第一，公共协议不根据算法名称编写分支。它只调用接口，并从组合配置读取容量和格式。第二，应用入口不直接操作陷门、矩阵或多项式；命令行和浏览器收到的都是统一角色对象和密文字节。因此，同一套页面、状态管理和错误处理可以服务不同组合。

## 5.2 稳定的原语接口

HIBE 后端提供五个密码方法,IBS 后端提供四个密码方法和一个签名加载方法。表 5.1列出接口及其返回对象。

**表 5.1** HIBE 与 IBS 后端接口

| 后端 | 方法 | 公共含义 |
|---|---|---|
| HIBE | setup | 建立公开参数与主秘密 |
| HIBE | extract | 为第一层接收身份生成密钥 |
| HIBE | derive | 为第二层发送身份委派叶密钥 |
| HIBE | encrypt | 沿两层身份路径加密固定长度种子 |
| HIBE | decrypt | 用叶密钥恢复种子或返回拒绝 |
| IBS | setup | 建立签名公开参数与主秘密 |
| IBS | key_extract | 为发送身份生成签名密钥 |
| IBS | sign | 签署规范协议消息 |
| IBS | verify | 以公开参数、身份和消息验签 |
| IBS | load_signature | 从规范字节恢复签名对象 |

每种公开参数、主秘密、用户密钥、密文和签名都实现规范的 to_bytes 与 from_bytes。对象头包含类别、版本和参数摘要。后端加载对象时检查字段数量、长度、数值范围和公开上下文，公共层无需了解其中的系数排列。

接口也明确随机源的传递方式。建立、发钥、加密和随机化签名从上层取得密码学随机源；验签与规范解码不读取随机数。测试可以注入可重复随机流，正常运行使用操作系统随机源。同一方法在不同入口下因此保持相同的随机性边界。

## 5.3 组合配置与接入流程

### 5.3.1 完整组合配置 SuiteConfig

一套运行配置由 SuiteConfig 描述。它记录 HIBE 与 IBS 选择器、算法版本、两边参数档、密钥和签名格式、外部身份长度、混合载荷预算以及协议版本。配置使用排序字段和规范 JSON 编码，其摘要进入公开对象和所有角色密钥。

注册表为每个后端保存四类信息：构造函数、参数预设、对象解码器和能力声明。能力声明包括两层身份支持、内部身份宽度、直接明文容量、IBS 输入上限、签名尺寸上界、所需运行依赖和允许的编码格式。发现后端和查看配置只读取这些静态信息，不生成密钥。

运行前的 validate_suite 依次完成：

1. 解析 HIBE 与 IBS 选择器，固定精确版本和参数档；

2. 检查 HIBE 支持两层身份，外部身份编码后不超过内部上限；

3. 计算 IBS 签名输入开销和最大应用消息长度；

4. 检查签名上界与应用消息能够放入混合载荷预算；

5. 检查对象格式和运行依赖，生成完整配置摘要。

只有配置通过以后，系统才进入 Setup。容量不足会在昂贵建立之前给出所需字节数和现有预算，用户可以选择更大载荷预算或另一后端。

### 5.3.2 新增 HIBE 或 IBS 的接入步骤

接入新 HIBE 时，实现者完成五个 HIBE 方法和五类对象，声明两层身份宽度、种子容量和参数预设，再向注册表提供一个后端描述。接入新 IBS 时，实现四个密码方法、签名加载和四类对象，声明身份、签名输入及签名输出上限。公共协议、Hybrid、BYTEAUTH、状态文件和应用入口均不修改。

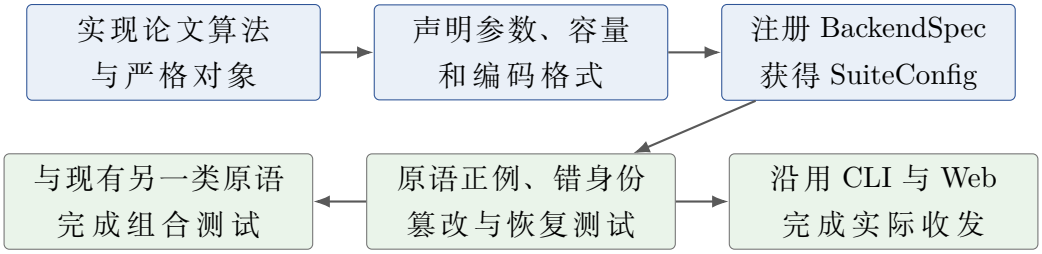



**图 5.2** 新增密码后端的接入过程

以 Wang–PW 为例，新增代码位于独立 IBS 目录，向外暴露与 Sageloli、SRGG 相同的方法。注册描述声明目标档 $s$0-4、393,320 字节签名和 4,096 字节签名输入上限。组合层据此算出 3,924 字节应用容量，并把混合载荷预算提高到能够容纳该签名的范围。随后直接复用 LATTE、五个 IB-ME 算法、命令行角色导入和错误输入流程，公共协议中没有新增“如果是 Wang 则……”的分支。

这一接入方式也是“通用设计”的软件依据。更换算法需要完成新原语本身及其元数据，而不是复制一套 IB-ME 实现。十种常规配对和 LATTE + Wang–PW 都经过同一入口形成。

## 5.4 固定发钥与持久状态

现实系统不会为每条消息重新建立主密钥。发钥机构需要长期保存主状态，并保证同一身份能够取回先前发布的密钥。作品为 HIBE 接收者和 IBS 发送者分别维护发钥记录。

一次发钥分为准备、生成和提交三个阶段。准备阶段取得本地锁并写入待处理身份；生成阶段调用底层原语；提交阶段先原子保存更新后的主状态，再发布用户密钥。若进程在生成中断，恢复程序会看到待处理记录并停止继续发钥，避免同一身份在状态不明时获得第二份对象。

加载状态时，系统按图 5.3检查组合配置、公开上下文、参数快照、对象类型和身份绑定。后端随后检查自己的代数关系，例如 LATTE 叶钥方程、SRGG 对象形状或 Wang–PW 的 $AX + H_1 = G$ 关系。通过后才恢复运行实例。

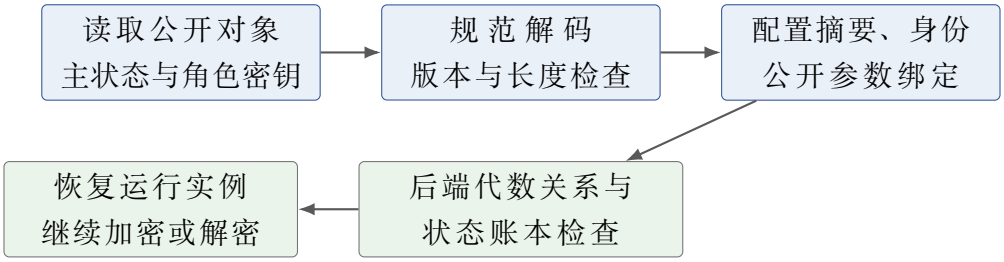



**图 5.3** 持久对象恢复流程

公开对象、发送密钥、接收密钥和密文可以分别导出。角色导入只要求该角色实际需要的对象：发送端不读取 HIBE 主秘密，接收端不读取 IBS 主秘密。这样可以在不同进程甚至不同主机上执行发送和接收，同时保持相同的组合配置与编码。

## 5.5 命令行与浏览器操作

命令行适合批量实验、持久状态和跨进程恢复。典型流程为：选择组合并建立系统；为 Alice、Bob 导出角色密钥；Alice 指定 Bob 并加密；Bob 指定 Alice 并解密；关闭进程后重新导入对象，再次解密已有密文。实验驱动在此基础上记录每阶段时间、对象字节数和进程树 RSS。

浏览器界面使用同一运行时，不另写一套密码代码。使用者首先选择组合，然后输入接收者、发送者、期望发送者和消息。页面显示建立、发钥、签名、封装和解密阶段，并展示密文长度、耗时和最终结果。对于耗时较长的 B 组，任务在服务端继续执行，页面刷新后可以重新连接正在运行的任务。




**图 5.4** A 组实际收发界面：指定双方身份、加密消息与匹配解密

图 5.4记录 A 组的一次实际操作：Alice 向 Bob 发送 36 字节 UTF-8 消息，Bob 选择 Alice 为期望发送者后恢复原文。A、B 还分别在同一条密文上执行三种身份错配、翻转 1 bit 和截短 1 字节，均得到拒绝。两组该次密文长度分别为 142,095 字节和 149,255 字节。

## 5.6 运行优化与部署方式

除了第 4 章三项主要优化，系统还减少两类重复工作。BNN 签名包含机构证书和用户消息签名。对同一身份的连续消息，系统缓存“证书已经验证”的公开结果，但每条消息仍重新验证用户签名。缓存绑定完整证书、用户公钥和身份，并使用有界 LRU 管理。

SRGG 签名的一部分数值准备只取决于固定签名密钥和数值选项。实现可以复用该计划，每条签名仍重新生成随机量。实验表明这两类局部缓存对完整流程的影响小于结构化采样、紧凑编码和 Lai 会话，因此它们作为运行细节保留。

系统支持单机演示和分角色部署。单机模式把发钥机构、发送端、接收端和浏览器服务放在一台主机，便于比赛现场操作。分角色模式由发钥机构离线生成角色对象，发送端和接收端分别导入自己的密钥；密文可通过任意字节传输通道送达。

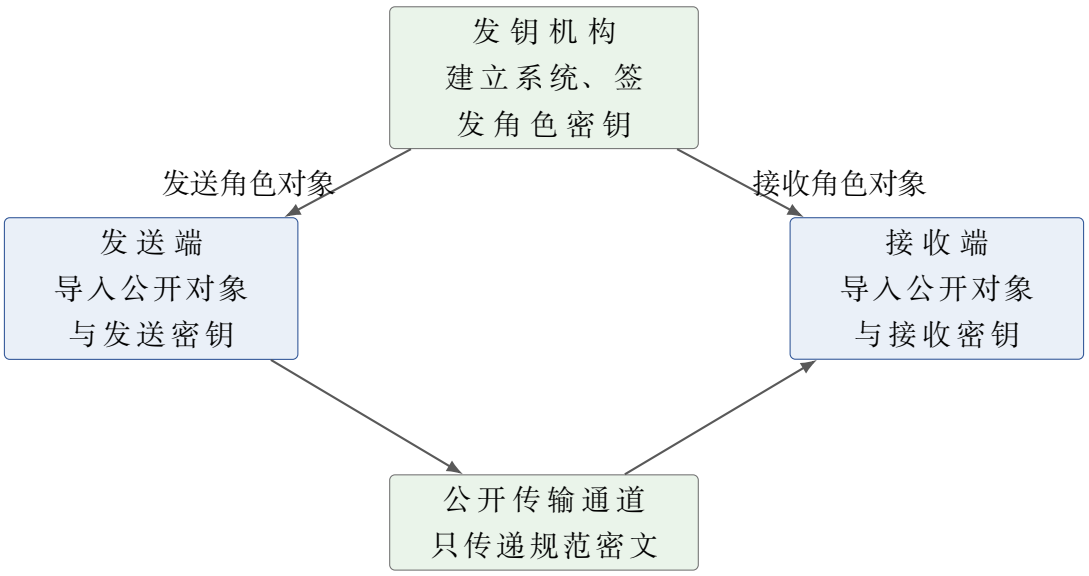



**图 5.5** 发钥机构、发送端与接收端的部署关系

原生模块在创建适配器时会检查接口和二进制。已有部署对照发现，把安装包和运行文件放在 Linux 本地文件系统可以减少跨挂载目录的重复读取。该调整不改变算法、参数和密文格式，第 6 章列出同一运行时下的对照结果。

除主要实验机器外，项目还在 Xeon Silver 4216 主机的 NixOS、Sage 10.9 和 Python 3.14.7 环境完成独立安装。A、B 各执行 25 条操作，C 完成匹配解密、内部签名拒绝和恢复。C 最初因 900 秒阶段上限导致超时；调整为 3600 秒后完成三个长阶段，耗时约 21 至 27 分钟。这次复现用于检查安装、对象恢复和完整调用路径；性能分析仍采用第 6 章逐项列明的原始实验环境。

## 5.7 本章小结

软件系统通过稳定接口、组合配置和注册表把底层算法与公共协议分开。新增后端只实现自身算法、对象与能力声明，随后即可复用五个 IB-ME 算法、状态管理、CLI 和浏览器入口。固定发钥和规范恢复使系统能够跨进程长期使用，单机与分角色部署则覆盖演示和实际通信两种方式。

# 第六章　测试与结果分析

## 6.1　实验设置

实验分别考察完整系统的运行成本、三项主要优化、部署改动和功能正确性。参数来源见第 2 章；身份和消息采用规范字节编码，正常密码操作使用操作系统随机源。不同实验的环境与重复方式集中列于表 6.1。

**表 6.1**　主要实验环境与重复方式

| 实验 | 环境 | 输入与重复 |
| --- | --- | --- |
| A/B 完整流程 | i9-14900HX, WSL2, Sage 10.7 / Python 3.12.13, OMP 4、BLAS 1 | 身份各 64 B，消息 32 B；每组独立建立 3 套系统，每套 3 对重复加解密 |
| Sageloli 采样 | Ultra 7 255HX, WSL2, Python 3.13.15, OMP 与 BLAS 均 4 | PARAM-I、II 各建立 1 套；身份 21 B、消息 33 B 及后续消息 |
| SRGG 编码 | Ultra 7 255HX, WSL2, Python 3.13.15, OMP 与 BLAS 均 1 | 每档 3 套系统；同对象交替测量两种编码各 5 次 |
| Lai 完整组合 | Ultra 9 275HX, WSL2, Sage 10.9 / Python 3.13.14, OMP 与 BLAS 均 1 | 身份各 64 B，消息 32 B；C 组合 1 次，解密进程存在并发 |
| 接收会话 | 与 A/B 主实验的机器及运行时相同 | 1 套系统；原解密 2 次，准备 1 次，会话解密 3 次 |
| 部署位置 | Ultra 9 275HX, WSL2, Python 3.13.14, OMP 4、BLAS 1 | 相同 3 套状态；每种位置各 30 次完整加密、解密 |

完整流程计时包含组合编码、对象检查和原语调用。首次解密包含叶钥生成；恢复实验另起进程加载已有对象。表中重复测量描述固定密钥下的运行波动，独立建立系统的数量单独列出。下文时间通常保留 3 位有效数字；B、KiB、MiB 分别表示字节、$2^{10}$ 字节和 $2^{20}$ 字节。

### 6.1.1　十种常规组合的覆盖范围

两种 HIBE 与五种常规 IBS 形成表 6.2所示的十种组合。每一格都使用真实后端完成系统建立、双方发钥、消息加密、双方身份匹配解密和错误身份拒绝。表格按底层算法计数；同一算法的不同参数档和参考 / 紧凑格式不重复计为新方案。

**表 6.2** 两种 HIBE 与五种常规 IBS 的端到端组合

| HIBE | BNN + ML-DSA | BNN + Falcon | Luo 模格 | SRGG | Sageloli |
|---|---|---|---|---|---|
| LATTE | 完成 | 完成 | 完成 | 完成 | 完成 |
| Lai | 完成 | 完成 | 完成 | 完成 | 完成 |

十种组合用于确认通用接口和完整协议覆盖。A、B、C 进一步承担定量分析：A、B 固定 LATTE，观察不同 IBS 带来的在线成本；C 同时更换 HIBE 和 IBS，观察高成本委派与环格签名。Wang–PW 不并入十格矩阵，它是在常规十种组合完成后接入的第六种 IBS，目前有 LATTE 一套目标档组合。

### 6.1.2 评价指标

实验按“离线准备、在线收发、持久恢复”三阶段记录。离线准备包括系统建立与双方发钥；在线收发包括首次加密、首次解密和固定密钥下的重复操作；持久恢复另起进程加载公开对象与角色密钥。时间采用墙钟时间，尺寸采用实际规范编码字节数，内存采用定期采样的进程树 RSS 峰值。

这些指标对应不同部署选择。发钥机构更关心 Setup、KeyExtract 和主状态，发送端更关心签名与完整加密，接收端更关心叶钥委派和解密，网络侧则关心完整密文而不是单个签名。后文先分析完整系统，再把总成本分解到具体优化。

## 6.2 完整系统的时间与存储成本

### 6.2.1 A/B 组合对比

A、B 使用相同 LATTE 参数与相同输入，主要差别在 IBS。表 6.3给出完整生命周期的中位时间，图 6.1展示重复加解密的各次测量。

**表 6.3** A/B 完整流程中位时间，单位为秒

| 操作 | A | B | 测量次数（A/B） |
|---|---|---|---|
| 系统建立 | 1.12 | 1.6 | 3/3 |
| 发送者发钥 | 0.0954 | 32.1 | 3/3 |
| 接收者发钥 | 0.427 | 0.457 | 3/3 |
| 首次加密 | 0.0963 | 9.14 | 3/3 |
| 首次解密 | 0.284 | 0.239 | 3/3 |
| 重复加密 | 0.0785 | 9.17 | 9/9 |
| 重复解密 | 0.152 | 0.161 | 9/9 |
| 对象恢复 | 0.845 | 0.89 | 3/3 |
| 恢复后解密 | 0.154 | 0.134 | 9/9 |

两组接收者发钥和重复解密接近，额外成本主要出现在 B 的发送者发钥与签名生成。A、B 重复加密中位数分别为 78.5 毫秒和 9.17 秒，重复解密分别为 152 毫秒和 161 毫秒。A 适合作为交互演示基线；B 展示原生模块格 IBS 在同一完整协议中的运行特点。

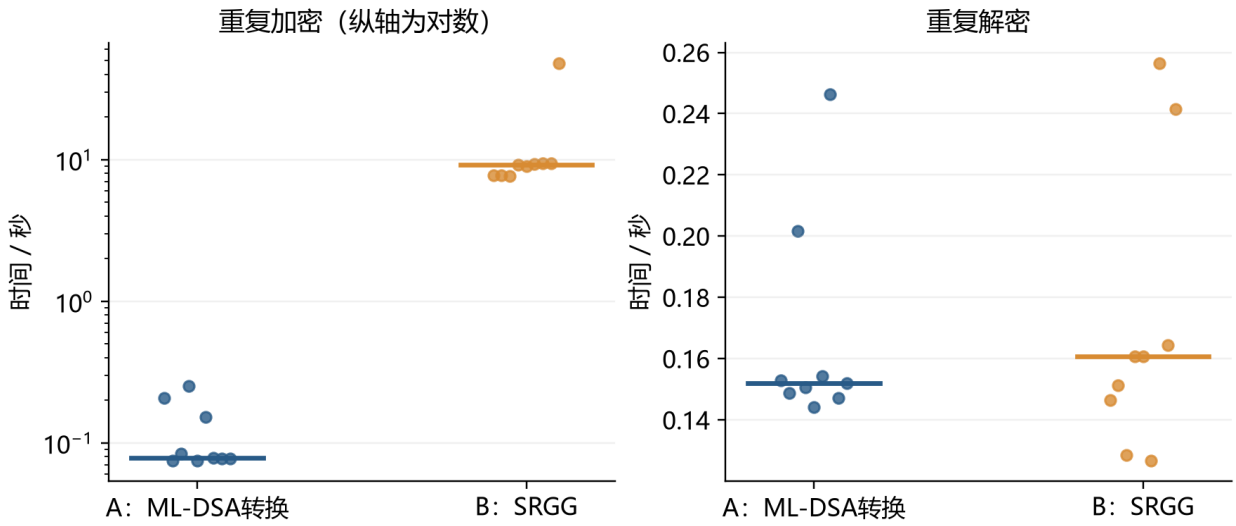



**图 6.1** A/B 重复操作的时间分布。每组 3 套系统、每套 3 次，横线为中位数；加密面板使用对数纵轴

各次测量还说明只看中位数会遗漏波动。B 有一次重复加密耗时约 48.2 秒，明显高于其中位数；A 也存在较慢观测。首次解密高于重复解密，与叶钥生成有关。恢复对象后，解密回到重复操作相近的量级。

### 6.2.2 对象尺寸与运行内存

表 6.4汇总公共对象、主状态和双方密钥的编码长度。A、B 完整密文约 139 KiB 和 146 KiB；本章后面的密文组成分析进一步解释这些字节来自哪里。

**表 6.4** A/B 对象编码长度约值与运行内存

| 组 | 公钥 | 主状态 | 发送钥 | 接收状态 | 完整密文 | 内存/MiB |
|---|---|---|---|---|---|---|
| A | 14 | 173 | 7.86 | 125 | 139 | 460 |
| B | 46.2 | 262 | 82.4 | 125 | 146 | 485 |

前五项单位为 KiB。对象长度按记录范围取近似值；内存为每 10 毫秒采样的进程树 RSS 最高值，包含运行库和子进程，短时峰值可能未被采样捕获。

B 的公钥、主状态与发送钥大于 A，接收状态接近，符合两组共用 HIBE 而采用不同 IBS 的结构。编码文件大小和运行内存分别统计：紧凑密钥文件加载后仍会恢复为数学对象，运行时还需要数值库、采样工作区和子进程。

### 6.2.3 C 组合的委派开销

在 C 的一次 32 字节消息组合测试中，加密耗时 99.0 秒，匹配解密 425 秒，其中 Lai 委派为 393 秒，占约 92.5%。这说明对固定通信双方，重复生成同一身份路径的叶钥是优先优化的位置。该次数据来自 Ultra 9 机器，解密进程存在并发，用于分析其阶段组成。

C 的传输密文约 4.34 MiB，大部分来自 Sageloli 签名和 Lai 胶囊。它展示了另一类 HIBE 与环 IBS 在论文配置下的完整组合，以及会话摊销的使用方式。下面分别分析通信量与准备成本。

### 6.2.4 Wang–PW 目标档的分阶段运行

Wang–PW 目标档使用 $n = 512$、$q = 4294967291$ 和 $s_0 = 4$。其密钥和采样工作区远大于常规组合，因此在 128 GiB 级服务器上完成目标规模运行。表 6.5汇总已经保存的建立、发钥、独立签名和 LATTE 组合结果。

**表 6.5** Wang–PW 目标档与 LATTE 组合的实测结果

| 项目 | 结果 | 说明 |
|---|---|---|
| Setup | 536.63 s | 生成目标档公开矩阵和根陷门 |
| 公开参数 | 67,109,371 B | 规范二进制对象 |
| 主状态 | 603,980,295 B | 保存紧凑根陷门与参数 |
| KeyExtract | 5,431.95 s | 为一个 64 字节身份生成用户陷门 |
| 用户签名密钥 | 2,214,593,131 B | 规范二进制对象 |
| 独立 Sign | 175.79 s、175.15 s | 两次恢复后的新消息签名均通过公开验签 |
| 签名 | 393,320 B | 包含身份盐、消息盐和短原像 |
| 签名峰值 RSS | 约 41.3 GiB | 进程树采样峰值 |
| LATTE + Wang 加密 | 192.54 s | 78 字节应用消息，包含一次新 Wang 签名 |
| 匹配解密 | 2.85 s | 从恢复的接收状态完成诚实解密 |
| 完整密文 | 529,207 B | Hybrid 与完整字节认证后的 wire 长度 |
| 组合驱动总时长 | 254.86 s | 含准备、一次加密、诚实解密和错误输入检查 |

目标档实验分阶段保存并复用已经生成的根和用户密钥。Setup、KeyExtract 来自较早的目标生命周期，最终两次签名与 LATTE 组合使用修订后的签名实现；表中按阶段呈现这些结果，不把它们相加成一次连续生命周期。

从使用过程看，Setup 和 KeyExtract 适合放在发钥机构的离线阶段。发送端一次签名约 176 秒，并需要约 41.3 GiB 峰值内存，是组合加密的主要成本。接收端只进行公开验签与 LATTE 解封装，诚实解密约 2.85 秒。393,320 字节 IBS 签名进入对称载荷后，完整密文约 517 KiB，明显小于用户签名密钥，但仍高于 A、B。

这一结果表明通用框架能够容纳大对象矩阵 SIS 后端，同时也说明它更适合低频签名、离线发钥和资源充足的发送端。三组代表保持 A、B、C 不变，Wang–PW 作为目标规模补充方案单独展示。

## 6.3 完整密文的通信开销

### 6.3.1 完整密文的组成

混合加密使应用消息不再受 HIBE 短明文容量直接限制，但种子胶囊、IBS 签名和一次性认证仍需要传输。依据现有安装运行记录与实际编码格式，可将 A 的一条 32 字节消息和 C 的一条空消息分解为表 6.6。

**表 6.6** 两条已有密文的字节组成核算

| 组成部分 | A：消息 32 B | C：空消息 |
|---|---|---|
| 一次性公钥 | 65,536 | 65,536 |
| 一次性签名 | 32,768 | 32,768 |
| IBS 签名 | 6,237 | 3,227,824 |
| HIBE 胶囊 | 37,088 | 1,228,844 |
| 应用消息 | 32 | 0 |
| 上下文、格式、nonce 与标签 | 432 | 432 |
| 合计 | 142,093 | 4,555,404 |

整包长度来自已有运行；固定字段按序列化模块核算，HIBE 胶囊长度由整包扣除其他组成得到。该核算没有新增加密实验。

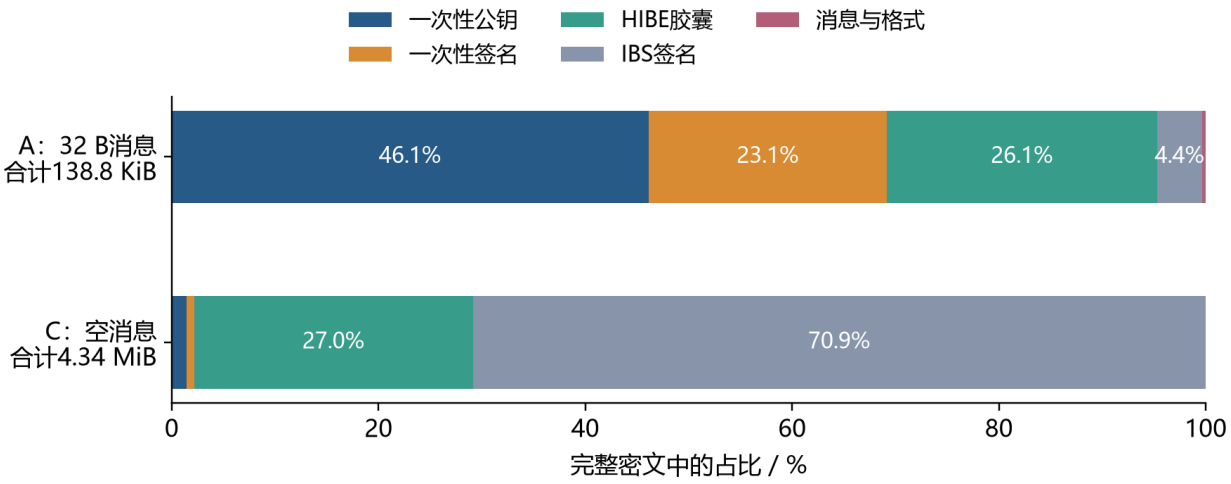



**图 6.2** 完整密文的主要成本来源：A 以一次性认证为主，C 以 IBS 签名和 HIBE 胶囊为主

Lamport 一次性公钥含 1,024 个 64 字节哈希像，签名含 512 个 64 字节原像，两项合计固定 96 KiB，约占这条 A 密文的 69.2%。因此，缩短几字节应用消息几乎不会改变 A 的整包长度。C 的 IBS 签名约占 70.9%，HIBE 胶囊约占 27.0%；其后续通信优化应优先考察这两部分。

这也解释了 SRGG 签名紧凑编码与完整 B 密文的关系：签名字段变短会减少对称载荷，但整包还包含与 A 相同的一次性认证结构和 LATTE 胶囊。评价编码时应同时看单体对象与完整协议中的占比。

### 6.3.2 消息长度与可用容量

安装运行测试已完成 A 的 0、32、1,024 和 59,191 字节消息，B 的 0、32、1,024 和 65,364 字节消息；超过应用上限 1 字节时，系统在生成新密文前拒绝。该批最大长度密文约为 197 KiB 和 210 KiB。

C 已完成空消息和 1,024 字节消息，两条密文相差 1,024 字节，符合固定胶囊与签名加上可变应用载荷的组成。C 的 3,924 字节上限来自容量计算，尚缺该最大长度的加密解密记录。上述容量检查使用签名保守上界，具体签名和胶囊的编码长度仍可能随随机输出变化。

## 6.4 三项主要优化的实验结果

### 6.4.1 Sageloli 结构化采样结果

Sageloli 两个参数档均完成建立、身份发钥、两次签名和验证。表 6.7给出各档一套系统的运行记录。

**表 6.7** Sageloli 结构化实现的实际运行，时间单位为秒

| 参数档 | 建立 | 发钥 | 首次签名 | 后续签名 | 验证 | RSS/GiB |
|---|---|---|---|---|---|---|
| PARAM-I | 4.99 | 85.1 | 9.28 | 2.02 | 0.517 | 1.36 |
| PARAM-II | 43.6 | 520 | 38.6 | 5.02 | 1.36 | 6.19 |

PARAM-I 发钥约 85.1 秒，记录峰值 RSS 约 1.36 GiB；PARAM-II 发钥约 520 秒，约 6.19 GiB。首次签名明显高于第二次，与同一身份陷门下数值准备的复用相符。结构化实现的主要成果是让这些参数下的采样与验证能够实际完成。稠密矩阵的 1.18 TiB 是第 4 章按表示方式计算的存储估算，实验记录对应结构化实现。

### 6.4.2 SRGG 紧凑编码结果

图 6.3和表 6.8展示三档 SRGG 相同数学对象的两种编码。每档独立建立 3 套系统，在同一对象上交替执行两种格式的编解码。

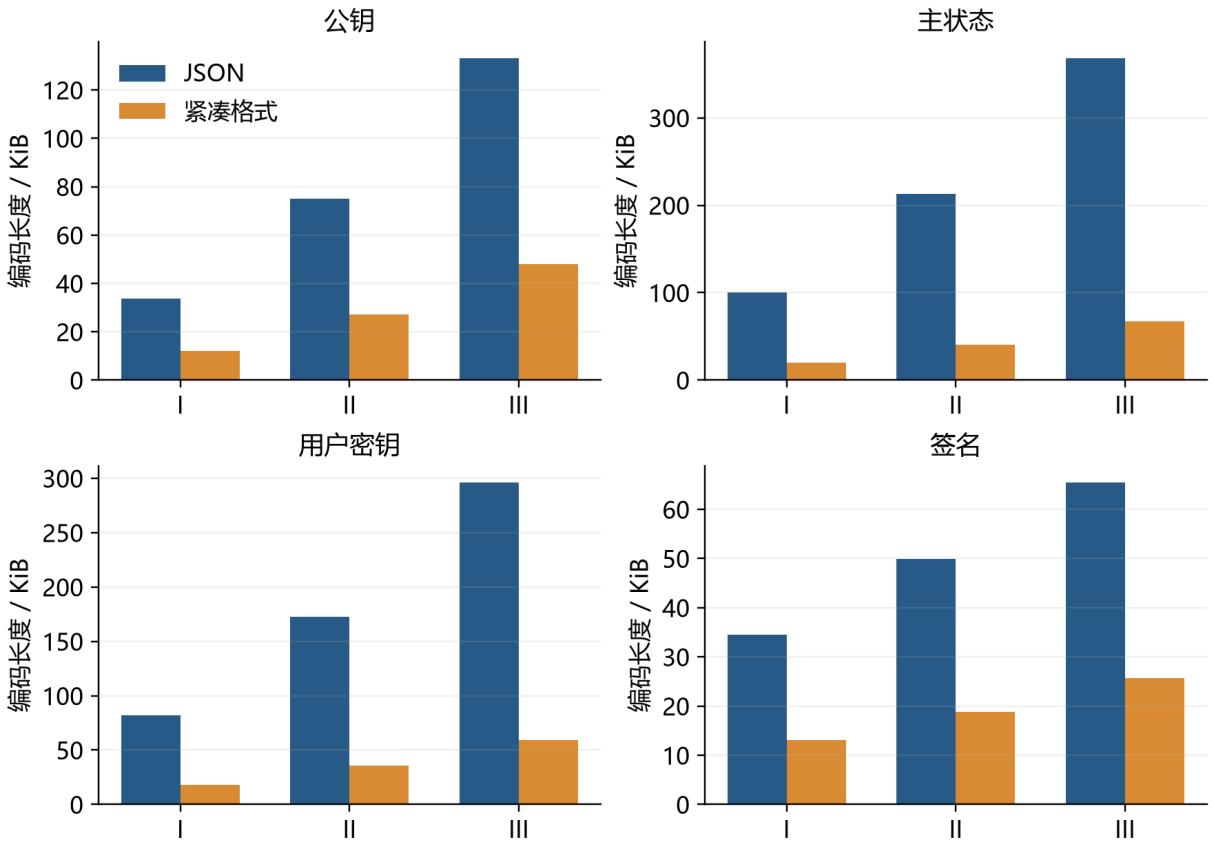



**图 6.3** SRGG 三档参数的四类对象尺寸；I、II、III 表示参数档

**表 6.8** SRGG 对象编码长度，单位为 KiB

| 参数档 / 格式 | 公钥 | 主状态 | 用户钥 | 签名 |
|---|---|---|---|---|
| I / JSON | 33.7 | 100 | 82.2 | 34.5 |
| I / 紧凑 | 12.1 | 19.8 | 17.8 | 13.1 |
| II / JSON | 75.2 | 213 | 173 | 49.9 |
| II / 紧凑 | 27.1 | 40.1 | 35.6 | 18.8 |
| III / JSON | 133 | 368 | 296 | 65.5 |
| III / 紧凑 | 48.1 | 67.5 | 59.5 | 25.6 |

公钥长度缩小约 2.77 至 2.79 倍，主状态 5.05 至 5.46 倍，用户钥 4.61 至 4.98 倍，签名 2.55 至 2.65 倍。主状态包含较多小范围陷门系数，去除十进制和结构符号开销后，缩减最明显。

紧凑格式需要位打包、取值检查和规范解码。例如 PARAM-I 主状态的紧凑编码与解码阶段中位时间约 23.1 和 61.3 毫秒，JSON 约 0.974 和 37.3 毫秒。该项优化的主要收益是对象存储与传输长度，适用于重视密钥保存和通信量的使用方式。

### 6.4.3 Lai 接收会话结果

Lai 会话实验使用一套主密钥，原路径解密 2 次，建立会话 1 次，会话内解密 3 次。原解密中位数为 526 秒，会话准备约 510 秒，后续解密中位数为 28.6 秒，额外保留叶钥编码 34.7 MiB。

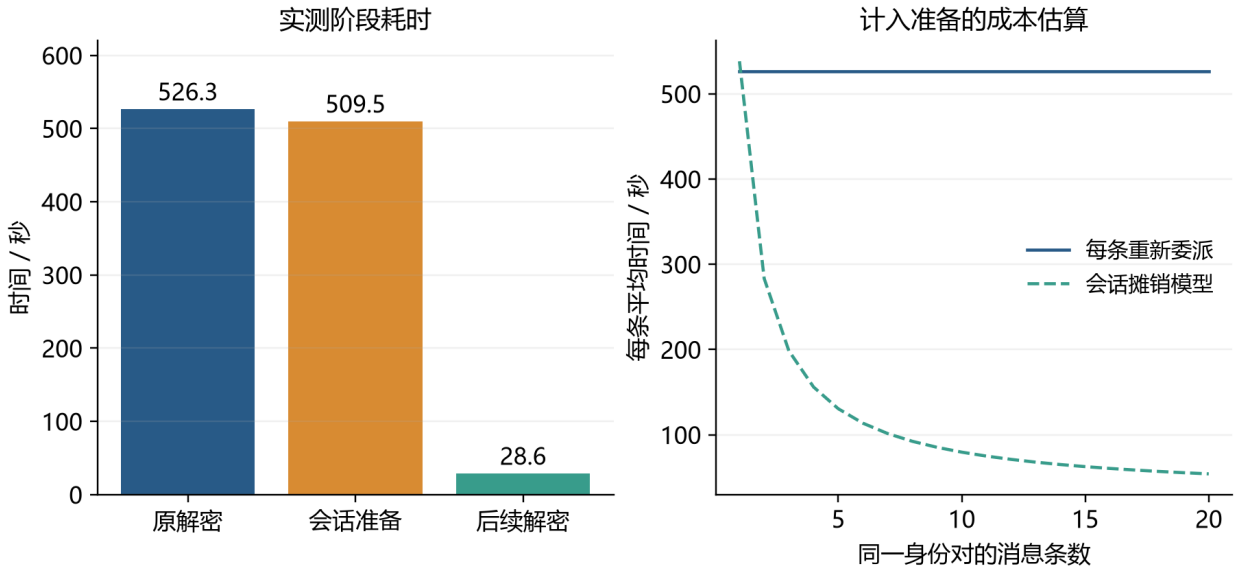



**图 6.4** Lai 接收会话：左图为测得阶段耗时，右图由阶段中位数代入成本公式估算

准备完成后的单条时间约为原路径的 1/18.4。把准备计入后，首条平均成本约 538 秒；按 $\bar{T}(N)$ 模型，处理 10 条和 100 条消息时约为 79.5 秒和 33.7 秒。右图使用已有阶段耗时外推消息数量，未逐条执行整条曲线。

会话方法把主要委派开销移到首次使用，适合固定身份对的连续消息。短暂的一次通信仍要承担完整准备；长期会话则以额外秘密状态换取较低的后续延迟。该组样本来自一套系统，后续扩大测量可进一步观察不同主密钥下的波动。

## 6.5 缓存与部署位置的完整流程对照

### 6.5.1 局部缓存对总耗时的影响

在 A、B 各 3 套系统中，分别开启和关闭优化，以相同输入执行完整流程。A 缓存机构证书的成功验证结果，原生验签调用由每条 2 次减少为 1 次，解密时间比中位数为 1.086，逐套范围 0.992 至 1.339。B 复用签名数值准备，加密时间比中位数 1.030，范围 1.009 至 1.921。

这些结果说明，减少一次局部调用后，总时间仍受其余密码运算与运行波动影响。A 有一套测量略微回退，B 的中位收益较小。因此，系统保留缓存容量与关闭选项，主要优化结论仍以结构表示、编码和 Lai 会话为重点。

### 6.5.2 文件部署位置对运行时间的影响

三路部署对照固定参数、状态和原生引擎，均由同一 WSL Python 运行，分别使用 Windows 盘源码、Windows 盘安装包及 Linux 盘同一安装包。每路完成 30 次完整加密和 30 次

解密，图 6.5给出中位数。

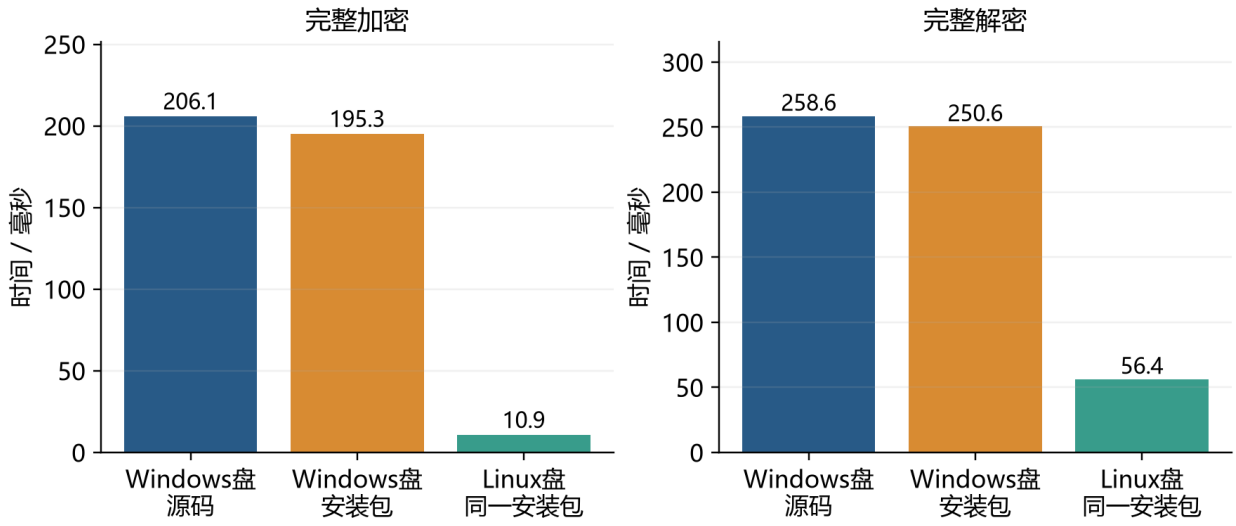



**图 6.5** 文件放置位置对完整 A 流程的影响；三路均使用同一 WSL 运行时

Linux 盘安装包相比 Windows 盘源码，加密与解密时间比分别约 18.87 和 4.59；只比较同一安装包的两种位置，时间比约 17.88 和 4.45。剖析显示，5 次加密的 515 次文件读取在挂载目录累计约 1.073 秒，在 Linux 本地约 0.011 秒。文件访问是该批运行的重要成本，本机部署优先采用 Linux 本地目录。

## 6.6 功能测试结果

三组均完成双方匹配恢复和三类身份错配拒绝。表 6.9汇总主要检查；另有 A 的随机身份与消息批次，独立建立 3 套系统、测试 12 组输入，60/60 项判断符合预期。

**表 6.9** 三组完整协议的主要功能结果

| 检查情况 | A | B | C |
|---|---|---|---|
| 双方身份匹配 | 恢复原文 | 恢复原文 | 恢复原文 |
| 仅接收身份不匹配 | 拒绝 | 拒绝 | 拒绝 |
| 仅期望发送身份不匹配 | 拒绝 | 拒绝 | 拒绝 |
| 双方身份都不匹配 | 拒绝 | 拒绝 | 拒绝 |
| 密文字节修改、截断、追加或重复字段 | 拒绝 | 拒绝 | 拒绝 |
| 外层认证有效、内部签名无效 | 拒绝 | 拒绝 | 拒绝 |
| 新进程加载对象后解密 | 成功 | 成功 | 成功 |

身份错配测试覆盖接收者、发送者和双方同时错误，分别触发不同层次的拒绝。密文字节修改、截断、追加和重复字段覆盖外层认证与规范解析；“外层认证有效、内部签名无效”

则单独检查 IBS 身份认证。新进程恢复测试重新加载公开对象和角色密钥，再对已有密文解密，覆盖持久对象的完整使用路径。

## 6.7 本章小结

实验展示了不同组合的成本分布。A 的在线收发最快，B 的主要时间集中在发送端 SRGG 签名，C 的主要时间集中在 Lai 叶钥委派；Wang–PW 的 Setup、发钥和签名需要服务器级资源，但接收端解密保持在数秒量级。三项主要优化分别降低了 Sageloli 采样内存、SRGG 对象尺寸和 Lai 连续解密延迟。

通信量分析表明，A 的密文主要由一次性认证构成，C 主要由 IBS 签名和 HIBE 胶囊构成。对象级优化能否明显缩短完整密文，取决于目标字段在整包中的占比。功能测试与恢复测试进一步确认，同一套协议在 A、B、C 以及十种常规组合中保持一致的身份匹配和字节处理流程。

# 第七章　总结与展望

## 7.1　工作总结

本作品围绕“接收者正确、发送者也正确”这一通信需求，完成了抗量子身份基匹配加密的通用设计、多算法实现和系统优化。发送者用身份指定接收对象，接收者在解密时指定认可的发送来源；两项身份同时匹配，系统才恢复消息。

协议以两层匿名 HIBE 控制身份路径，以 IBS 认证发送者。32 字节随机种子解决 HIBE 短明文与大签名、长消息之间的容量矛盾；HKDF 和 AES-256-GCM 承担应用载荷；一次性公钥承诺与外层签名把内部身份认证连接到完整密文字节。建立、双方发钥、加密和匹配解密五个算法在所有组合中保持一致。

软件层将 HIBE、IBS、组合配置、公共协议、状态管理和应用入口分开。两种 HIBE 与五种常规 IBS 形成十种端到端配对，覆盖 NTRU 结构、普通矩阵格、环格和模格路线。目标规模 Wang–PW 进一步与 LATTE 完成端到端组合。新增算法通过接口、规范对象和能力声明接入，不需要复制公共协议。

系统已经提供命令行、浏览器演示、固定发钥和跨进程恢复。使用者可以从算法选择开始，完成系统建立、角色密钥生成、消息发送、身份匹配解密和错误输入测试。公开对象、角色密钥和密文均采用带版本的规范编码，可以在不同进程间传递。

## 7.2　创新点及实际效果

作品的创新不只体现在接入算法数量，更体现在如何把数学方案变成可运行、可扩展的完整系统。表 7.1按照“问题—方法—结果—代价”归纳主要工作。

**表 7.1** 主要创新点、结果与使用取舍

| 问题 | 方法 | 结果 | 代价或适用方式 |
| --- | --- | --- | --- |
| 不同 HIBE / IBS 难以直接组合 | 独立接口、SuiteConfig、能力预检、短种子 Hybrid 与统一字节认证 | 十种常规配对和 LATTE + Wang–PW 复用同一五算法 | 新后端需实现严格对象并声明容量 |
| Sageloli 稠密展开不可承受 | 保留环多项式，逐频率执行低秩平方根与采样计划 | PARAM-I、II 均完成建立、发钥、签名和验证；RSS 约 1.36、6.19 GiB | 需要数值变换计划和精确方程复核 |
| SRGG 大整数对象存储冗长 | 按字段范围使用定长、Zigzag、二元位打包 | 主状态缩小 5.05–5.46 倍，签名缩小 2.55–2.65 倍 | 编解码增加少量处理时间 |
| Lai 固定身份对反复委派 | 接收会话预先生成并验证叶密钥 | 准备约 510 秒后，后续解密中位数 28.6 秒 | 额外保存约 34.7 MiB 会话叶钥 |
| 目标规模矩阵 SIS 对象巨大 | 流式对象、独立有限 Gaussian、固定发行与角色导入 | Wang–PW n512 完成签验、恢复和 LATTE 组合 | 用户钥约 2.06 GiB，签名约 176 秒、峰值约 41.3 GiB |

**通用设计**。通用性首先来自密码边界清晰。HIBE 只负责两层身份路径和种子封装，IBS 只负责发送身份认证，组合层只使用公开方法和对象字节。LATTE、Lai 以及六种 IBS 可以保留各自的模数、维度、环和采样方式。容量预检把原语差异提前转化为可计算的应用消息上限。

**结构化计算**。Sageloli 优化从环方程本身出发，不把多项式机械展开为巨型矩阵。PARAM-I 的相关扰动维数达到 403,456，完整实矩阵估算约 1.18 TiB；逐频率计算把核心分解降为 $65 \times 65$ 小矩阵，并在系数域继续执行精确验签关系。

**对象级优化**。SRGG 编码针对四类对象逐字段确定范围。同一数学对象可以在参考 JSON 与紧凑二进制之间无损往返。主状态中大量三值陷门系数只占 2 bit，签名中的挑战只占 1 bit，从而把存储收益落到实际密钥文件和密文载荷。

**使用模式优化**。 Lai 会话区分了“固定身份对的准备”和“每条消息必须执行的认证解密”。它没有减少每条消息的验签、GCM 和字节认证，而是消除同一路径的重复委派。这一方法特别适合机构内长期连接和固定设备对。

## 7.3 实验结论

A、B 固定同一 LATTE HIBE，因此能够直接观察 IBS 差异。A 的重复加密与解密中位数分别为 78.5 毫秒和 152 毫秒，适合交互演示与高频收发。B 的重复加密约 9.17 秒，时间主要花在 SRGG 签名；重复解密约 161 毫秒，与 A 接近。该结果说明，B 的主要成本在发送端而不是接收端。

C 更换为 Lai C2 与 Sageloli PARAM-I。一次完整解密约 425 秒，其中 Lai 委派约 393 秒。接收会话把固定路径的委派移到准备阶段后，后续解密中位数降至 28.6 秒。C 的完整密文约 4.34 MiB，主要由 Sageloli 签名和 Lai 胶囊组成。

Wang–PW 目标档展示了另一种资源分布。Setup 约 536.63 秒，单用户发钥约 5,431.95 秒；复用已发用户密钥后，一次签名约 176 秒，LATTE 组合加密约 192.54 秒，匹配解密约 2.85 秒。发送端的签名和用户密钥是主要开销，接收端仍能在数秒内完成公开验签与解封装。

密文组成分析还指出一个与算法选择同样重要的问题：A 组 96 KiB 固定空间来自 Lamport 一次性公钥和签名，占 32 字节消息密文的约 69.2%。C 组则由大尺寸 IBS 签名和 HIBE 胶囊主导。后续缩短通信量时，应根据组合选择不同目标，而不是只压缩应用消息。

## 7.4 后续工作

下一批实验将在 Intel Core Ultra 9 275HX 同一台机器、同一运行时和统一输入下，对十种常规组合各补测 10 组独立状态。测量将覆盖 Setup、双方发钥、首次与重复收发、恢复、对象尺寸和 RSS，并把首轮、热态和恢复后操作分开统计。完成后，实验章将用这组矩阵作为十种组合的统一横向结果，A、B、C 专项实验继续用于解释优化过程。

通信方面，A 组最值得继续研究的是替换或压缩当前 96 KiB 的一次性认证结构，同时保持内部身份签名与完整密文字节之间的绑定。C 组可以进一步压缩 Sageloli 签名和 Lai 胶囊，并增加多套独立主密钥下的会话测量。Wang–PW 则适合继续优化用户陷门表示、流式签名准备和发钥资源调度。

应用方面，可以把当前本地发钥机构扩展为独立服务，由业务系统完成账户审核和身份字符串映射，发送端与接收端只持有各自角色对象。现有规范编码、SuiteConfig 和角色导入已经提供了这一部署方式所需的基础。

总体而言，本作品已经从通用密码构造走到多原语、完整协议和可操作软件。不同格结构的方案能够在同一系统中选择和比较，主要计算与存储瓶颈也有对应的实现方法和实验结果。这些工作为后续参数完善、统一补测和面向具体场景的部署提供了稳定基础。

# 参考文献

[1] Peter W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[2] Adi Shamir. Identity-Based Cryptosystems and Signature Schemes. In *Advances in Cryptology—CRYPTO 1984*, LNCS 196, pp. 47–53, 1985.

[3] Yuejun Wang, Baocang Wang, Qiqi Lai, and Yu Zhan. Identity-based Matchmaking Encryption with Stronger Security and Instantiation on Lattices. *Theoretical Computer Science*, 2024. doi: 10.1016/j.tcs.2024.115048.

[4] Raymond K. Zhao, Sarah McCarthy, Ron Steinfeld, Amin Sakzad, and Máire O'Neill. Quantum-safe HIBE: Does It Cost a LATTE? *IEEE Transactions on Information Forensics and Security*, 2023.

[5] Russell W. F. Lai, Henry K. F. Cheung, and Sherman S. M. Chow. Trapdoors for Ideal Lattices with Applications. In *Inscrypt 2014*, LNCS 8957, pp. 239–256, 2015. doi: 10.1007/978-3-319-16745-9_14.

[6] Huiwen Jia, Ying Liu, Chunming Tang, and Lin Wang. Constructing Efficient Identity-Based Signatures on Lattices. *IET Information Security*, 2025. doi: 10.1049/ise2/6684889.

[7] Éric Sageloli, Pierre Pébereau, Pierrick Méaux, and Céline Chevalier. Shorter and Faster Identity-Based Signatures with Tight Security in the (Q)ROM from Lattices. In *ACNS 2023*, 2023. doi: 10.1007/978-3-031-33488-7_24.

[8] Jiaxin Pan and Benedikt Wagner. Short Identity-Based Signatures with Tight Security from Lattices. In *PQCrypto 2021*, 2021. ePrint 2021/970.

[9] David Galindo, Javier Herranz, and Eike Kiltz. On the Generic Construction of Identity-Based Signatures with Additional Properties. In *ASIACRYPT 2006*, 2006.

[10] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, 2024.

[11] Daniele Micciancio and Chris Peikert. Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In *EUROCRYPT 2012*, 2012.

[12] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. *Journal of the ACM*, 60(6), 2013.

[13] Adeline Langlois and Damien Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75, pp. 565–599, 2015.

[14] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.

[15] National Institute of Standards and Technology. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. Special Publication 800-38D, 2007.

[16] Leslie Lamport. Constructing Digital Signatures from a One Way Function. SRI International, CSL-98, 1979.

[17] Pingbin Luo, Xinjian Chen, Willy Susilo, and Qiong Huang. Identity-Based Signature from Lattices without Trapdoors. In *ICICS 2024*, 2024.

[18] Pierre-Alain Fouque et al. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Specification v1.2, 2020.